

第六章

软件总体设计

本章要点

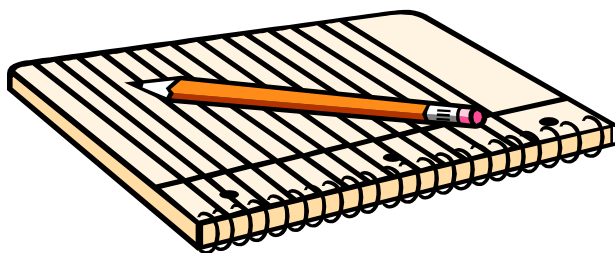
- **软件总体设计**

- **软件模块化设计**

- **软件结构与优化**

- **总体设计工具**

- **结构化总体设计**



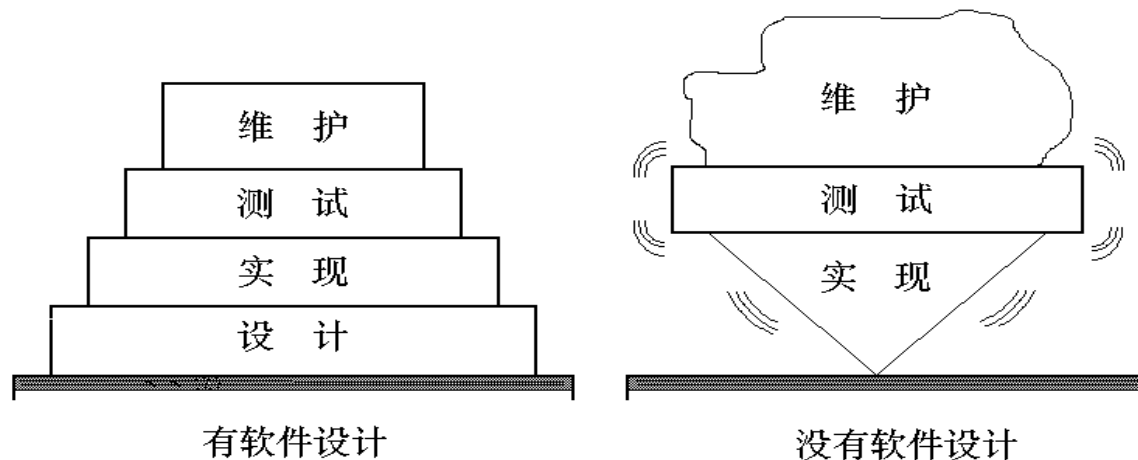
6.1 软件设计

6.1.1 软件设计介绍

- 设计阶段是确定软件“怎么做”。软件设计是一个迭代的过程，通过软件设计将用户的需求变为实现软件的“蓝图”。最初，蓝图只描述软件的整体框架，随着设计活动的深入，后续对软件的描述不断细化，形成一个可以实施的软件设计文件。
- 软件设计的目标是对将要实现的软件系统的体系结构、系统的数据、系统模块间的接口以及所采用的算法给出详尽的描述。

软件设计的重要性

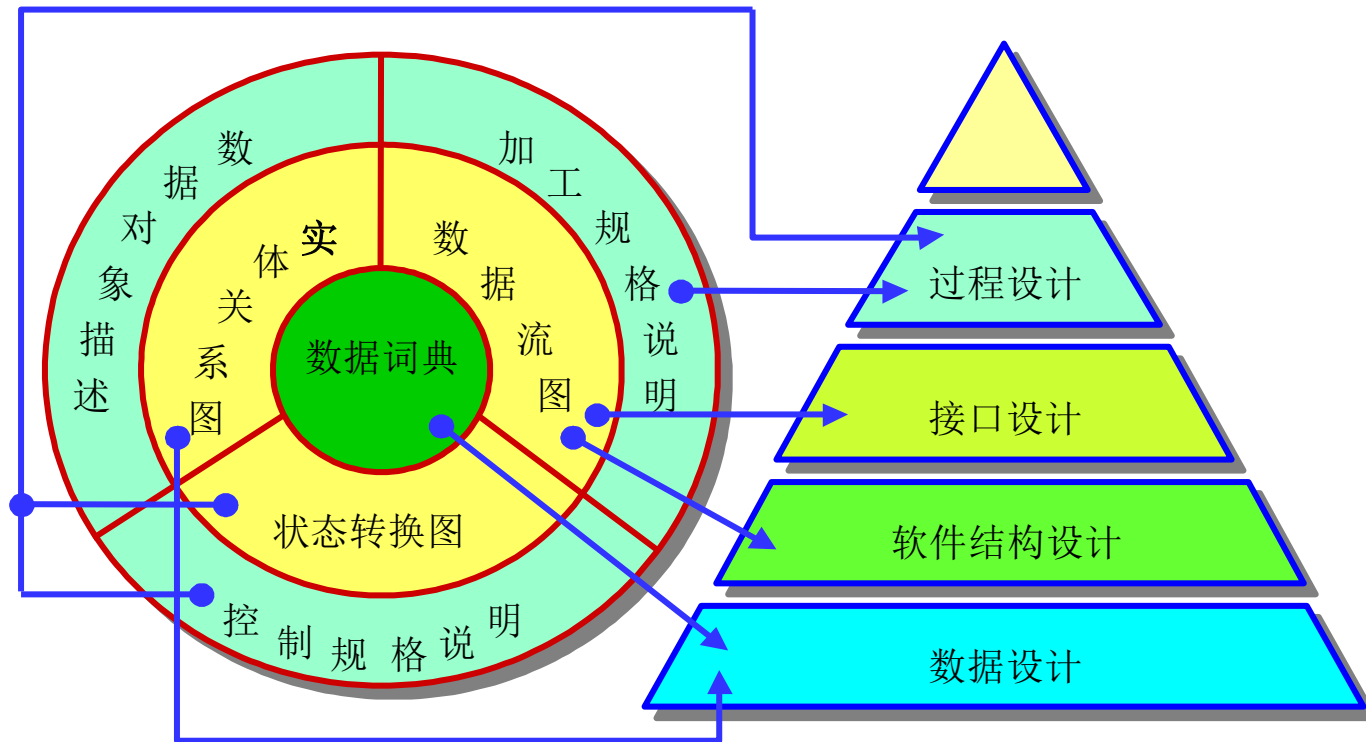
- 软件设计是开发阶段中最重要的步骤，它是软件开发过程中质量得以保证的关键步骤
- 设计提供了**软件的表示**，使得软件的质量评价成为可能。如果没有设计，只能建立一个不稳定的系统结构。



软件设计的目标和任务

- 根据用数据、功能和行为模型表示的软件需求，采用某种设计方法进行数据设计、软件结构设计、接口设计和过程设计。
- **数据设计**将实体-关系图中描述的对象和关系，以及数据词典中描述的详细数据内容转化为数据结构的定义。
- **软件结构设计**定义软件系统各主要成份之间的关系。
- **接口设计**根据数据流图定义软件内部各成份之间、软件与其它协同系统之间及软件与用户之间的交互机制。
- **过程设计**则是把结构成份转换成软件的过程性描述。

将分析模型转换为软件设计



软件设计任务

- 从工程管理的角度来看，软件设计分两步完成。
- **总体设计**，也称为结构设计、概要设计或高层设计。软件结构设计主要是仔细地分析需求规格说明，研究开发产品的模块划分，形成具有预定功能的模块组成结构，表示出模块间的控制关系，并给出模块之间的接口。软件结构设计中的输出是模块列表和如何连接它们的描述。
- **详细设计**，也称为(模块)过程设计或低层设计。详细设计是为结构设计中的各个模块设计过程细节，确定模块所需的算法和数据结构等。

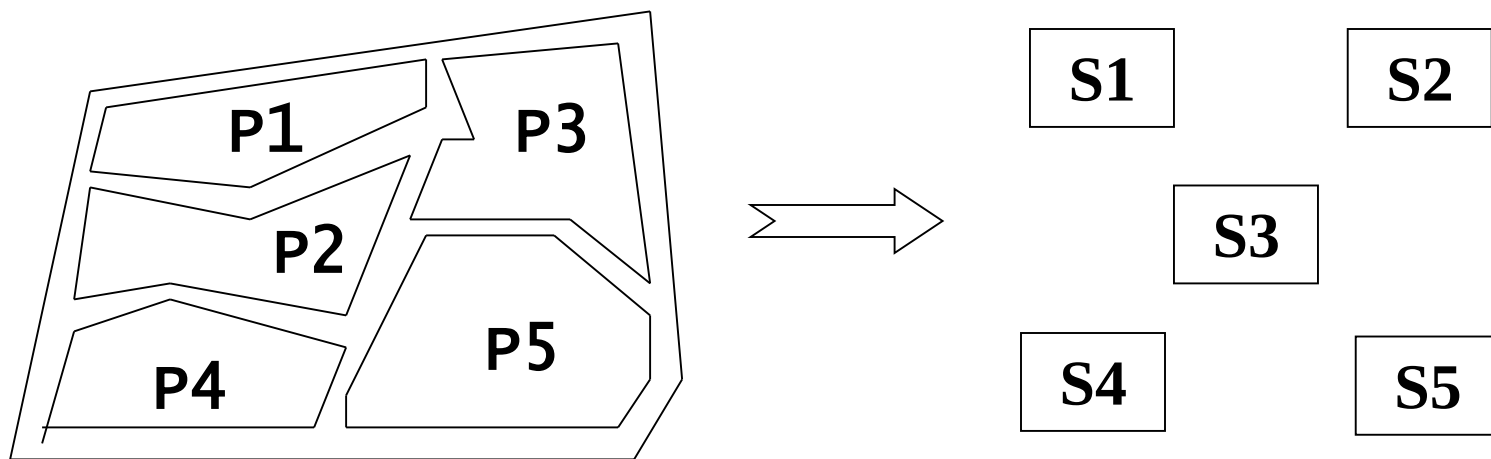
6.1.2 总体设计的步骤

- 设想供选择的方案（低、中、高）
- 选取审查合理的方案（参照可行性与投资）
- 功能分解和设计软件结构（使功能显而易见）
- 数据库设计
- 制定测试计划（测试计划从需求阶段开始）
- 编制设计文档
- 审查和复查

6.2 软件模块化设计

6.2.1 软件模块

- 一个大软件，由于其控制路径多、涉及范围广、变量多及其总体复杂性，使其相对于一个较小的软件不容易被人们理解。
- 把一个大而复杂的问题分解成一些独立的易于处理的小问题，解决起来就容易得多



软件求解的“问题”

模块化

- 把问题 / 子问题的分解与软件开发中的系统 / 子系统或者系统 / 模块对应起来，就能够把一个大而复杂的软件系统划分成易于理解和实现的模块式的结构。
- 模块化就是把程序划分成若干模块的过程，每个模块完成一个子功能，把这些模块集合起来组成一个整体，可以完成指定的功能满足问题的要求。

模块化的依据

设函数 $C(x)$ 定义问题 x 的复杂程度, 函数 $E(x)$ 确定问题 x 需要的工作量。

对于两个问题 $P1$ 和 $P2$,

如果: $C(P1) \gg C(P2)$

显然: $E(P1) > E(P2)$

根据人类解决问题的经验, 一个有趣的规律是:

$$C(P1+P2) > C(P1) + C(P2)$$

综合考虑得:

$$E(P1+P2) > E(P1) + E(P2)$$

模块

- 模块是数据说明、可执行语句等程序对象的集合，它是单独命名的而且可以通过名字来访问。如：过程、函数、子程序、宏等，模块又称为组件。
- 它一般具有如下三个基本属性：
 - 功能：**描述该模块实现什么功能
 - 逻辑：**描述模块内部怎么做
 - 状态：**该模块使用是的环境和条件

模块的特性

- 在描述一个模块时，还必须按模块的**外部特性**与**内部特性**分别描述

- **模块的外部特性**

模块的模块名、参数表、其中的输入参数和输出参数，以及给程序以至整个系统造成的影响

- **模块的内部特性**

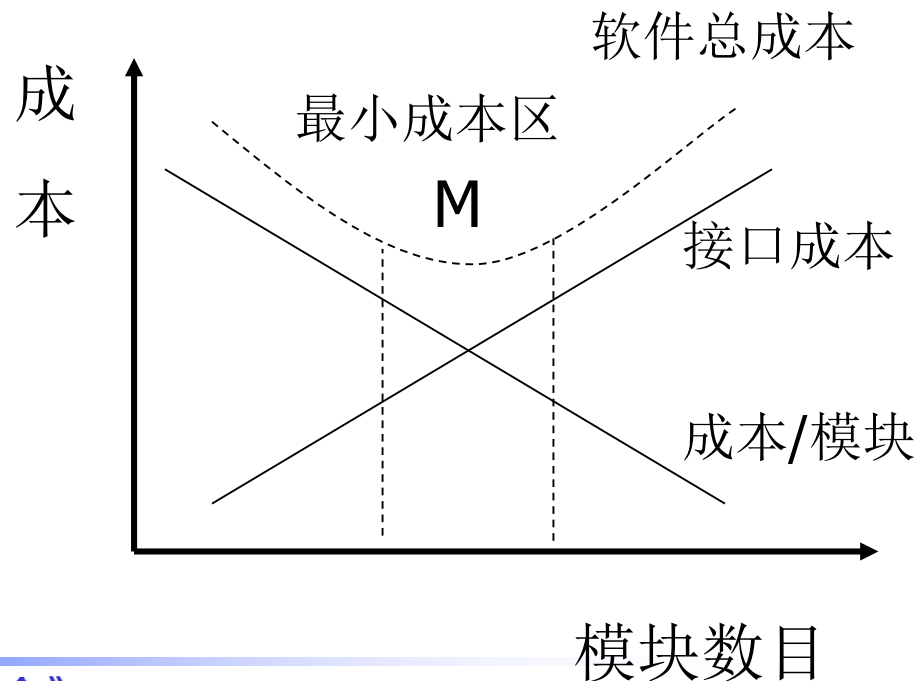
完成其功能的程序代码和仅供该模块内部使用的数据

- 对于模块的外部环境来说，只需要了解这个模块的外部特性足够了，不必了解它的内部特性。

模块大小、模块数目与费用的关系

- 模块化的好处:

一方面，模块化设计降低了系统的复杂性，使得系统容易修改；另一方面，模块化推动了系统各个部分的并行开发，从而提高了软件的生产效率。



模块大小、模块数目与费用的关系

- 如果模块是相互独立的，当模块变得越小，每个模块花费的工作量越低；但当模块数增加时，模块间的联系也随之增加，把这些模块联接起来的工作量也随之增加。

6.2.2 抽象

- 在考虑一个复杂问题时,最自然的办法就是分解,分解成的小问题就容易分析了。但是,分解需要抽象的支持。
- 在现实世界中一定事物、状态或过程之间总存在着某些相似的共性,把这些相似的方面集中和概括起来,暂时忽略它们之间的差异,这就是抽象。抽象是抓住主要问题,隐藏细节,这样才能容易分解。
- 抽象是抓住主要问题,隐藏细节,这样才能容易分解。

过程抽象

- 对软件进行模块设计的时候，可以有不同的抽象层次。
- 在软件工程过程中，从系统定义到实现，每进展一步都可以看做是对软件解决方案的抽象化过程的一次细化。
- 在软件需求分析阶段，用“问题所处环境的为大家所熟悉的术语”来描述软件的解决方法。
- 在软件计划阶段，软件被当做整个计算机系统中的一个元素来看待。
- 而在从概要设计到详细设计的过程中，抽象的层次逐次降低，当产生源程序时到达最低的抽象层次。

数据抽象

- 对实际的人、物、事和概念进行人为处理，抽取所关心的共同特性，忽略非本质的细节，并把这些特性用各种概念精确地加以描述。
- 数据抽象与过程抽象一样，允许设计人员在不同层次上描述数据对象的细节。
- 抽象是人类解决复杂问题的基本方法之一。只有抓住事物的本质，才能准确分析和处理问题，找到合理的解决方案。

6.2.3 逐步求精

- 逐步求精或称逐步细化,是一种自顶向下的设计策略。
- 将软件的体系结构按自顶向下方式, 对各个层次的过程细节和数据细节逐层细化, 直到用程序设计语言的语句能够实现为止, 从而最后确立整个的体系结构。
- 逐步求精是一个细化的过程, 从高抽象级上定义的功能陈述或数据描述开始, 然后在这些原始陈述上持续丰富越来越多的细节, 直至形成程序设计语言编写的语句。

抽象与求精

- **抽象和求精是一对互补的概念。** 抽象使得设计人员能够明确说明过程和数据而同时忽略低层细节。
- 求精实际上是细化过程，求精在设计过程中揭示出低层细节。

6.2.4 信息隐藏

- **信息隐藏原则指出：**应该这样设计和确定模块，使得每个模块的实现细节对于其它模块来说是隐蔽的。就是说，模块中所包含的信息（包括数据和过程）对于不需这些信息的模块来说，是不能访问的。
- 由于一个软件系统在整个软件生存期内要经过多次修改，所以在划分模块时要采取措施，使得大多数过程和数据对软件的其它部分是隐蔽的。这样，在将来修改软件时偶然引入错误所造成的影响就可以局限在一个或几个模块内部，不致波及到软件的其它部分。

6.2.5 模块独立性

- 模块独立性是模块化、抽象、信息隐蔽等概念的直接结果，也是模块化结构是否合理的标准。
- 模块独立性，是指软件系统中每个模块只涉及软件要求的具体的子功能，而和软件系统中其它的模块的接口是简单的。
- 例如，若一个模块只具有单一的功能且与其它模块没有太多的联系，则称此模块具有模块独立性。

模块独立性

- 模块独立性体现了有效的模块化，有两大优点：
 - 1. 独立性高的模块化易于开发和项目管理；
 - 2. 独立性高的模块容易测试和维护
- 模块独立性是一个良好设计的关键，而良好设计又是决定软件质量的关键。一般采用两个准则度量模块独立性，即模块间的耦合性和模块的内聚性。

模块独立的度量标准：内聚和耦合。

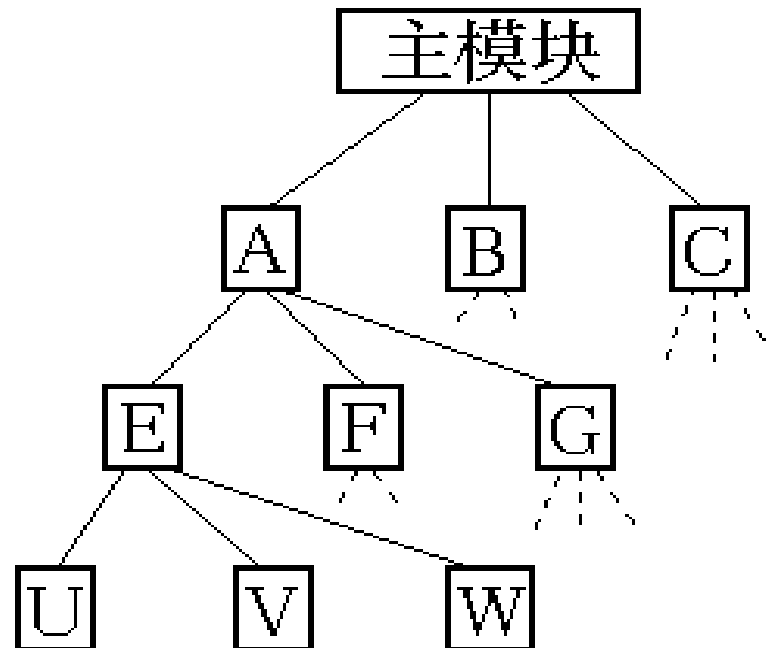
耦合与内聚

- **耦合**是**模块之间**的互相连接的紧密程度的度量。
- **内聚**是模块功能强度(一个**模块内部**各个元素彼此结合的紧密程度)的度量。
- 模块独立性比较强的模块应是**高内聚、低耦合**的。
- 耦合性取决于各个模块之间接口的复杂程度、调用模块的方式以及哪些信息通过接口调用。

(1) 非直接耦合

- 非直接耦合 (Nondirect Coupling) 指两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的

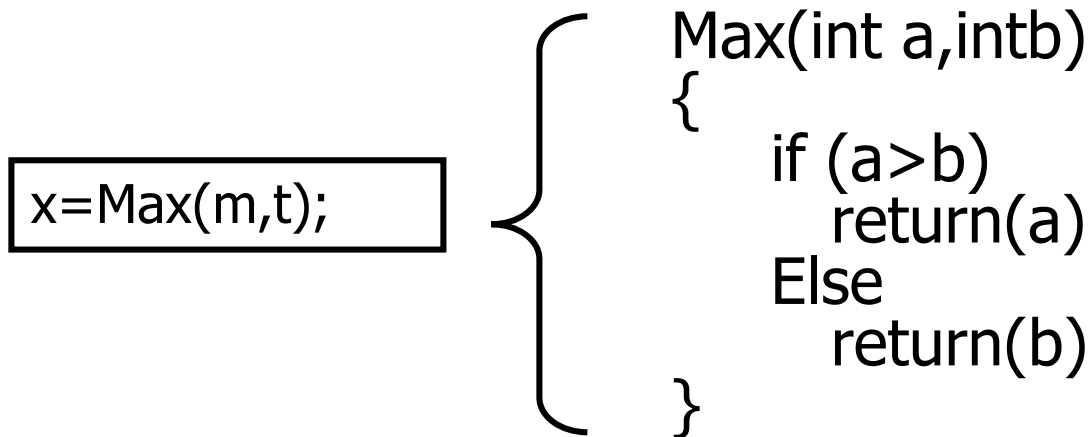
- 非直接耦合的模块
独立性最强



(2) 数据耦合

- 数据耦合 (Data Coupling) 指一个模块访问另一个模块时，只是通过参数交换信息，且交换的信息只是数据（不是控制参数、公共数据结构或外部变量）。

- 数据耦合是最低程度的耦合。

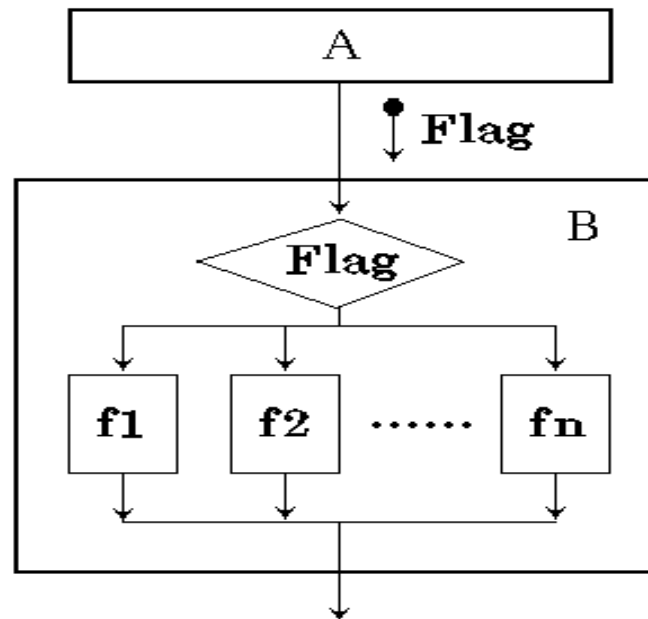


(3) 标记耦合

- 标记耦合(Stamp Coupling)指模块间通过参数传递复杂的数据结构，就是标记耦合。
- 如高级语言的数组名，记录名，文件名等这些名字即为标记，其实传递的是这个数据结构的地址，而不是简单变量。

(4) 控制耦合

- 控制耦合(Control coupling)指如果一个模块通过传送开关、标志、名字等控制信息, 明显地控制选择另一模块的功能, 就是控制耦合。
- 这种耦合的实质是在单一接口上选择多功能模块中的某项功能。

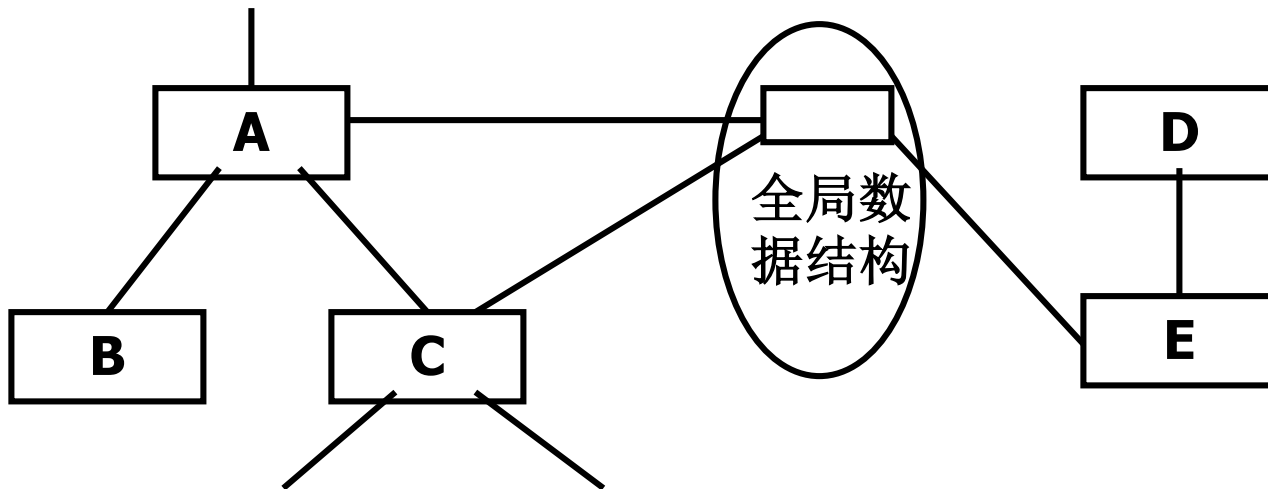


(5) 外部耦合

- 外部耦合 (External Coupling) 指一组模块都访问同一全局简单变量而不是同一全局数据结构，而且不是通过参数表传递该全局变量的信息，则称之为外部耦合。
- 例如C语言程序中各个模块都访问全局简单变量。

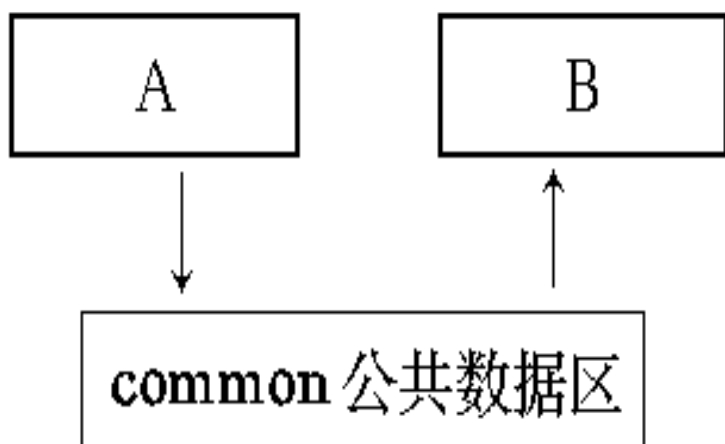
(6) 公共耦合

- 公共耦合(Common Coupling)指若一组模块都访问同一个公共数据环境，则它们之间的耦合就称为公共耦合。公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等。

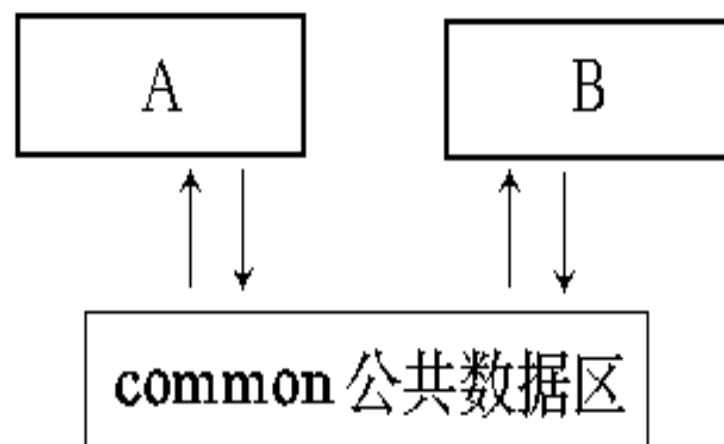


公共耦合

- 公共耦合的复杂程度随耦合模块的个数增加而显著增加。若只是两模块间有公共数据环境，则公共耦合有两种情况。松散公共耦合和紧密公共耦合。



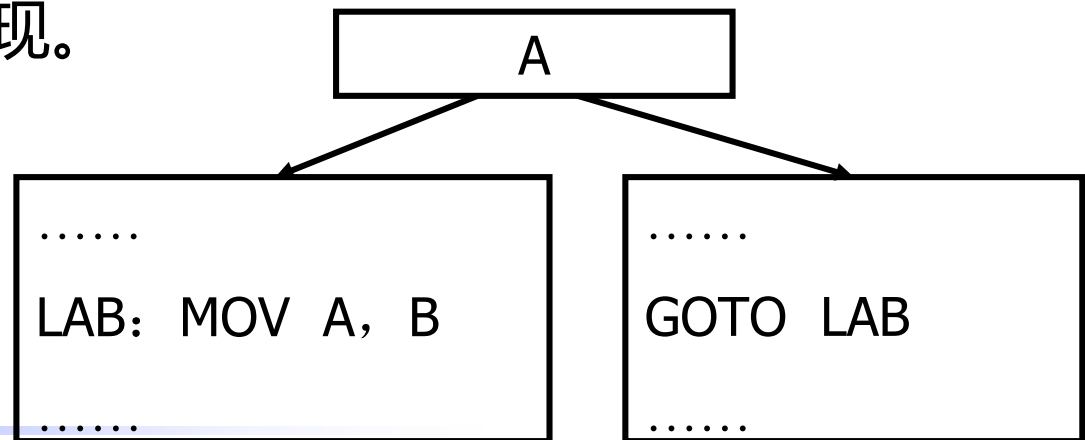
(a) 松散的公共耦合



(b) 紧密的公共耦合

(7) 内容耦合

- 内容耦合(Content Coupling)指一个模块直接访问另一个模块的内部数据；或者一个模块不通过正常入口转到另一模块内部；或者两个模块有一部分程序代码重迭；或者一个模块有多个入口，则两个模块之间就发生了内容耦合。
- 这种耦合也称为“病态耦合”，是模块独立性最弱的耦合，应避免出现。



模块间的耦合

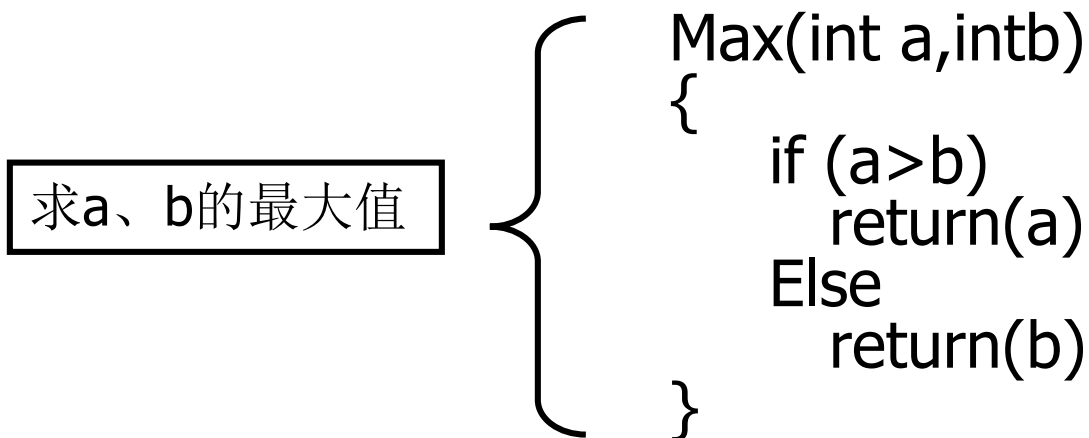


2、内聚

- 内聚是模块功能强度（一个模块内部各个元素彼此结合的紧密程度）的度量。一个内聚程度高的模块（在理想情况下）应当只做一件事。
- 一般模块的内聚性分为七种类型。

(1) 功能内聚

- 一个模块中各个部分都是为完成一项具体功能而协同工作，紧密联系，不可分割的。则称该模块为功能内聚模块。功能内聚模块是内聚性最强的模块。



- 在软件结构中，并不是每个模块都能归结为完成一个功能而设计成一个功能内聚的模块。

(2) 顺序内聚

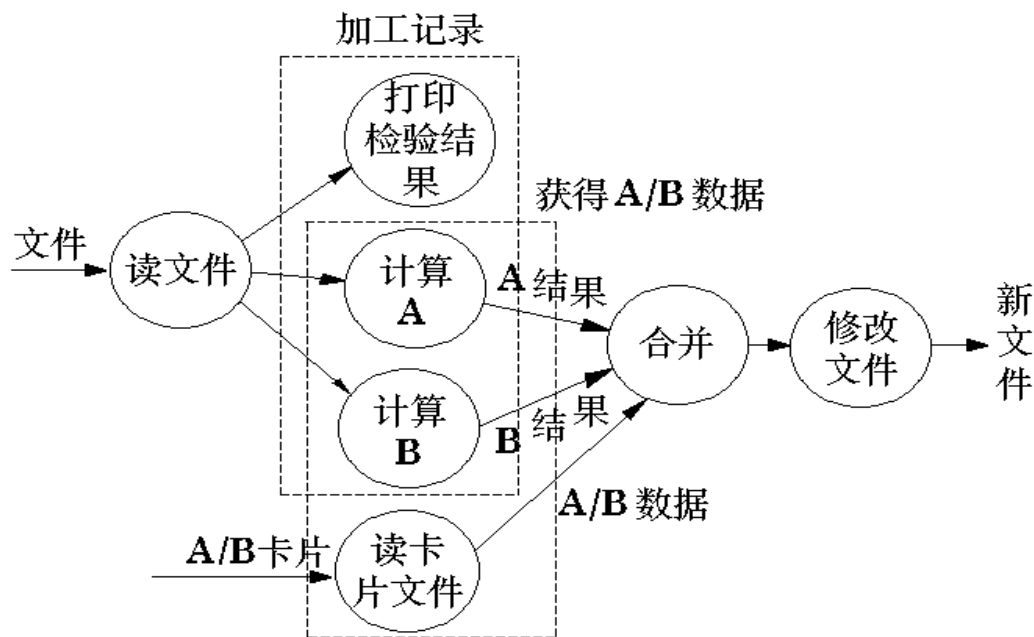
- 顺序内聚 (Sequential Cohesion) 指如果一个模块内处理元素和同一功能密切相关，而这些处理元素必须顺序执行，称为顺序内聚。通常一个处理元素的输出是另一个处理元素的输入。

求一元二次方程
根模块

- 
- A large right-facing curly bracket groups the three steps of the process, indicating they are sequential and related to the module.
- 1、输入系数
 - 2、求解
 - 3、打印方程的解

(3) 通信内聚

- 通讯内聚 (Communicational cohesion) 指如果一个模块内各功能部分都使用了相同的输入数据，或产生了相同的输出数据，则称之为通信内聚模块。
- 通常，通信内聚模块是通过数据流图来定义的。



(4) 过程内聚

- 过程内聚 (Procedural Cohesion) 指在使用流程图做为工具设计程序时，常常通过流程图来确定模块划分。把流程图中的某一部分划出组成模块，就得到过程内聚模块。这类模块的内聚程度比时间内聚模块的内聚程度更强一些。
- 例如，把流程图中的循环部分、判定部分、计算部分分成三个模块，这三个模块都是过程内聚模块。

过程内聚

- 过程内聚中的处理元素必须以特定的次序执行。其各组成功能由控制流联结在一起，实际是若干个处理功能的公共过程单元。
- 过程内聚和顺序内聚的区别：
 - ✓ 顺序内聚中是数据流从一个处理元流到另一个处理元。
 - ✓ 过程内聚中是控制流从一个动作流到另一个动作。

(5) 时间内聚

- 时间内聚 (Temporal Cohesion) 又称经典内聚。这种模块大多为多功能模块，但要求模块中的各个功能必须在同一时间段内执行。如初始化模块和终止模块，处理故障模块。
- 时间内聚模块比逻辑内聚模块的内聚程度又稍高一些。

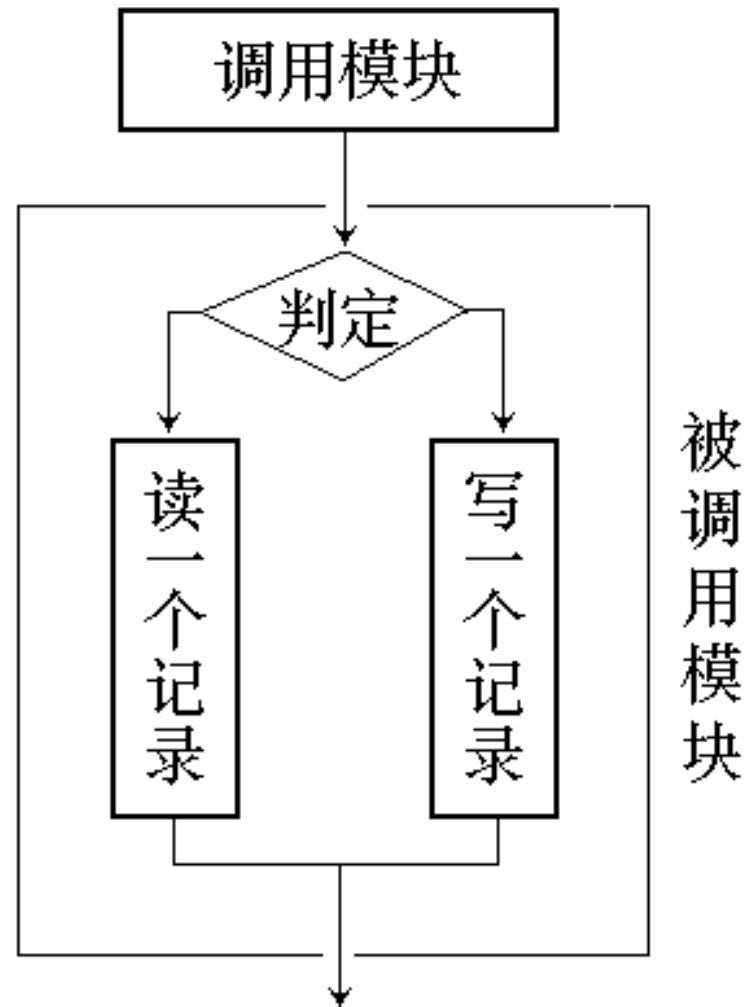
紧急故障处理模块

- 1、关闭文件
- 2、报警
- 3、保留现场

(6) 逻辑内聚

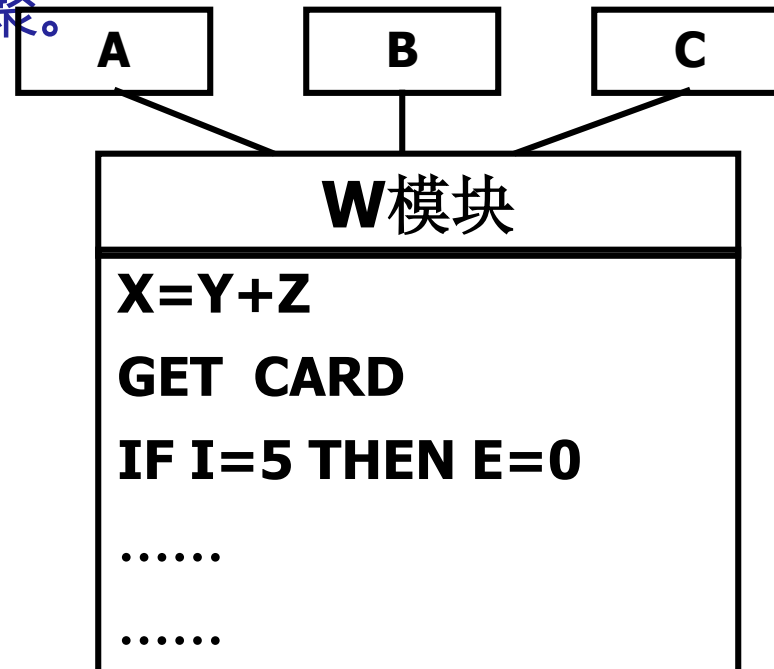
- 逻辑内聚 (Logical Cohesion)指

模块把几种相关的功能组合在一起，每次被调用时，由传送模块的判定参数来确定该模块应执行哪一种功能。



(7) 巧合内聚

- 巧合/偶然内聚 (Coincidental Cohesion) 指如果一个模块由完成若干毫无关系 (或关系不大) 的功能处理元素偶然组合在一起的, 称为巧合内聚。巧合内聚是最差的一种内聚。



模块内聚



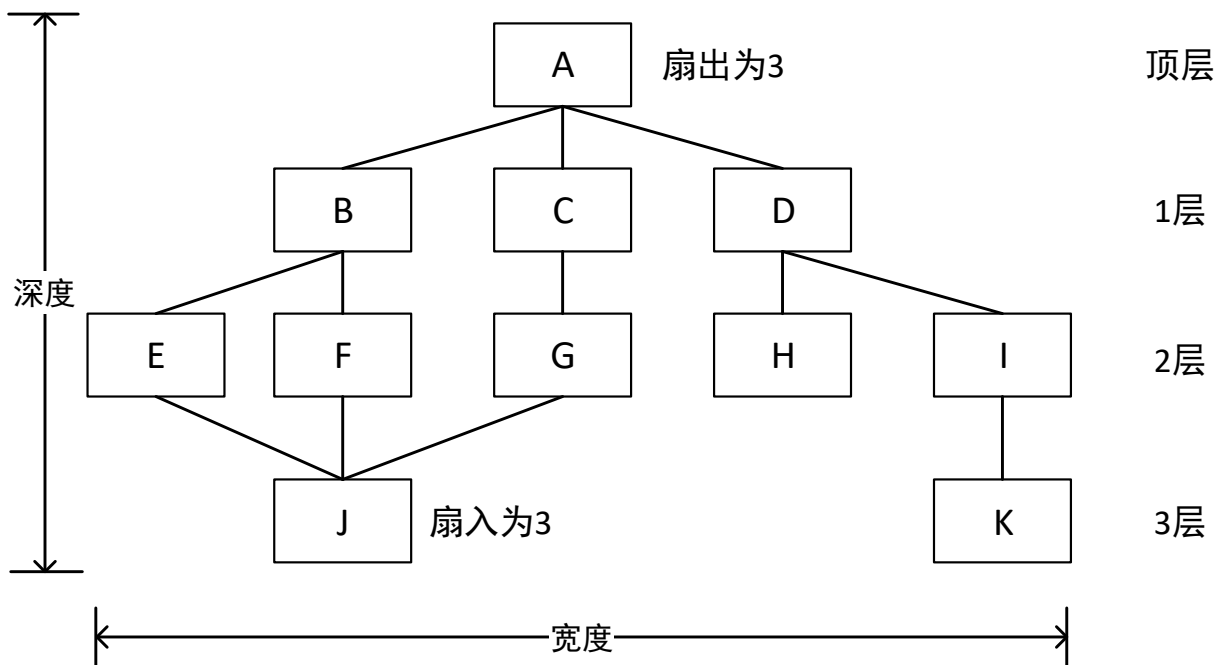
模块设计独立性

尽量使用数据耦合，少用控制耦合，限制公共环境耦合，完全不用内容耦合。

力求高内聚，通常中等程度的内聚也是可以的，效果和高内聚差不多，但低内聚不要使用。

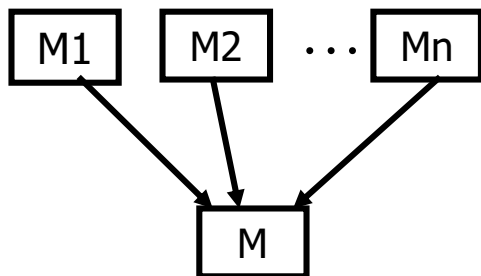
6.3 软件结构与优化

- 软件结构就是构成软件的模块的组织方式。模块之间可以有各种关系。一般可表示为层次结构和网状结构两种。

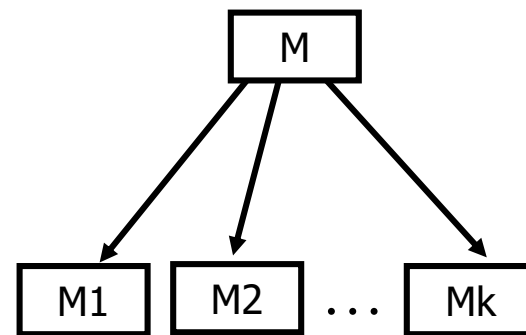


1. 层次结构

- 衡量模块层次结构的有关指标
- 深度--软件结构中控制的层数，粗略标志系统的大小和复杂程度；
- 宽度--软件结构内同一层次上的模块总数的最大值；
- 扇出--一个模块直接控制的模块数目；典型系统的平均扇出为3或4。扇出实际上是对问题解的分解。
- 扇入--有多少个上层模块直接调用它；



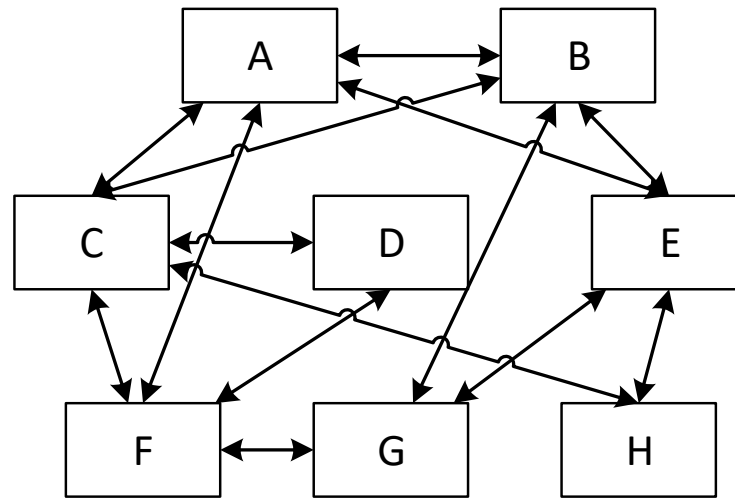
模块的扇入



模块的扇出

2 模块的网状结构

- 在网状结构中，任何两个模块都可以有双向的关系。由于不存在上级模块和下属模块的关系，也就分不出层次来。



- 对于不加限制的网状结构，由于模块相互关系的任意性，使得整个结构十分复杂，处理起来势必引起许多麻烦，这与原来划分模块为便于处理的意图相矛盾。

6.3.2 模块设计优化

- 软件的模块化设计除了要达到“正确”目标外，还必须进行优化，以满足性能要求和质量要求。优良的模块化设计往往又能导致程序设计的高效。

(1)改进软件结构提高模块独立性。

- 通过模块的分解和合并，力求降低耦合提高内聚。

模块设计优化

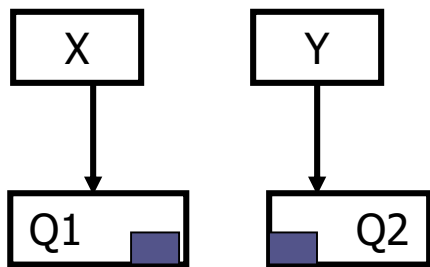


图1

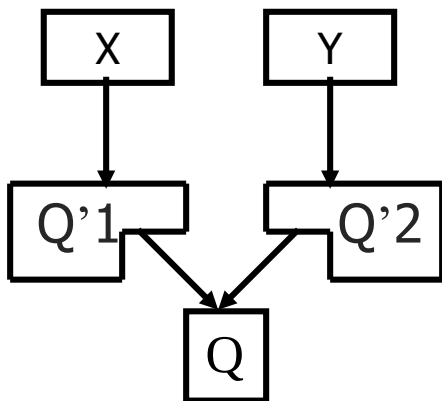


图2

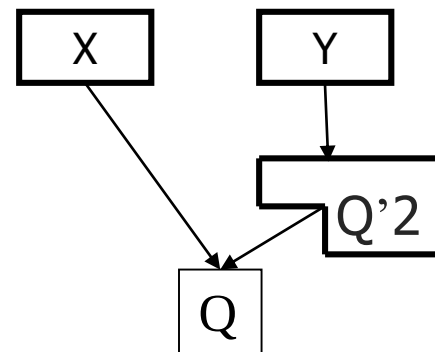


图3

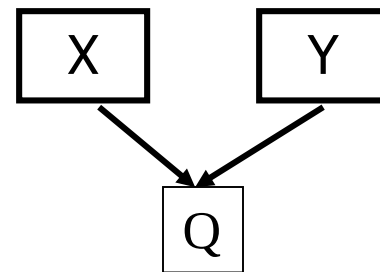


图4

图1，Q2中与已有的Q1相似；

图2，把Q1和Q2中相同的部分Q分离出来；

图3，若Q'1很小，可并入X中；

图4，若Q'2也很小，也可并入Y中；

分解模块

模块设计优化

(2) 在满足模块化要求的前提下尽量减少模块数量，在满足信息需求的前提下尽可能减少复杂的数据结构。

(3) 模块规模应该适中

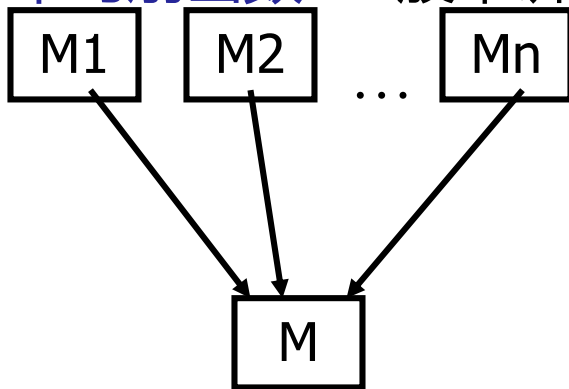
- ✓ 一个模块的规模不应过大，过大的模块往往分解不充分，但进一步分解不应该降低模块独立性。
- ✓ 防止模块功能过分局限。过小的模块开销大于有效操作，模块数目过多将使系统接口复杂。模块功能过分局限，使用范围过分狭窄，缺乏灵活性和可扩充性。

模块设计优化

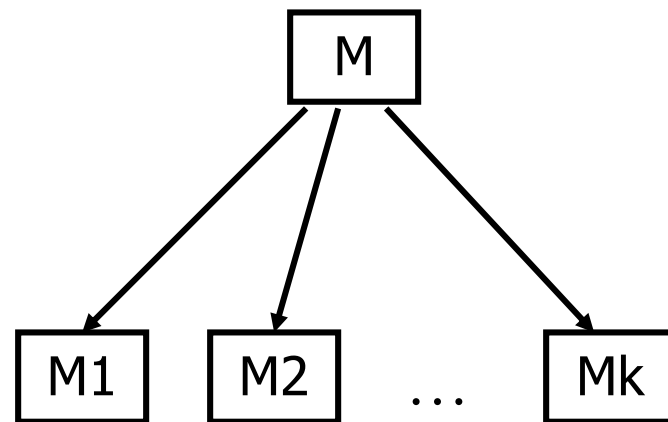
- (4) 深度、宽度、扇出和扇入都应适当

宽度数量 应控制在 7 ± 2 (普遍认为的人类"智力"限度值)

平均扇出数 一般来讲是3~5



模块的扇入

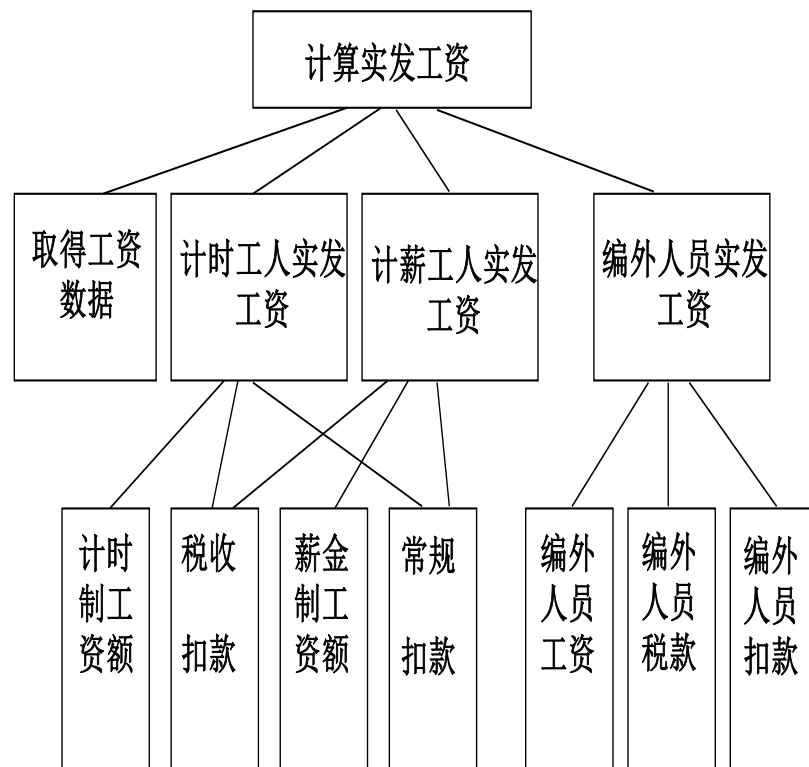
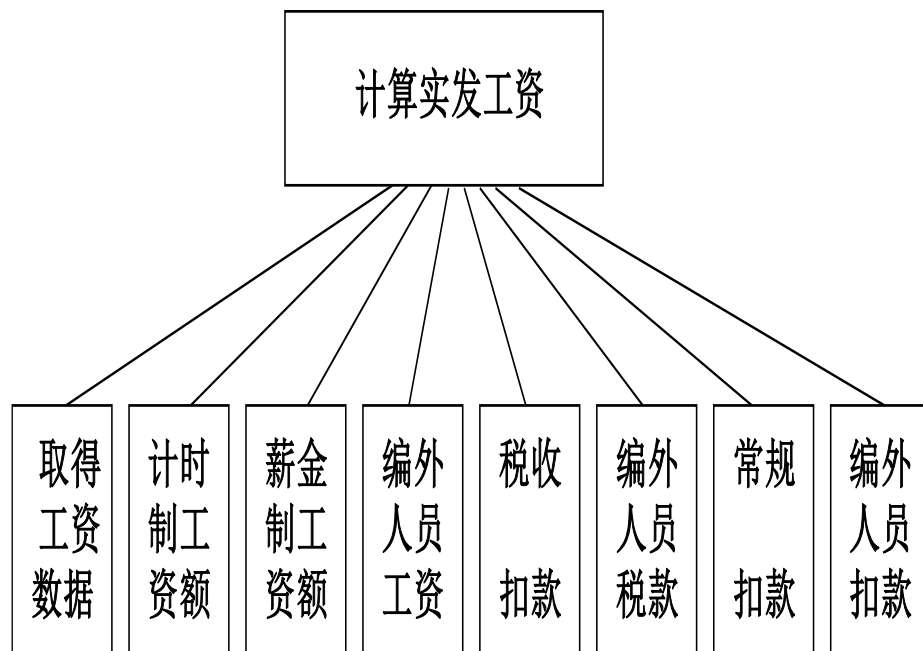


模块的扇出

模块设计优化

● 优化原则：减少高扇出，争取高扇入

增加中间层模块：

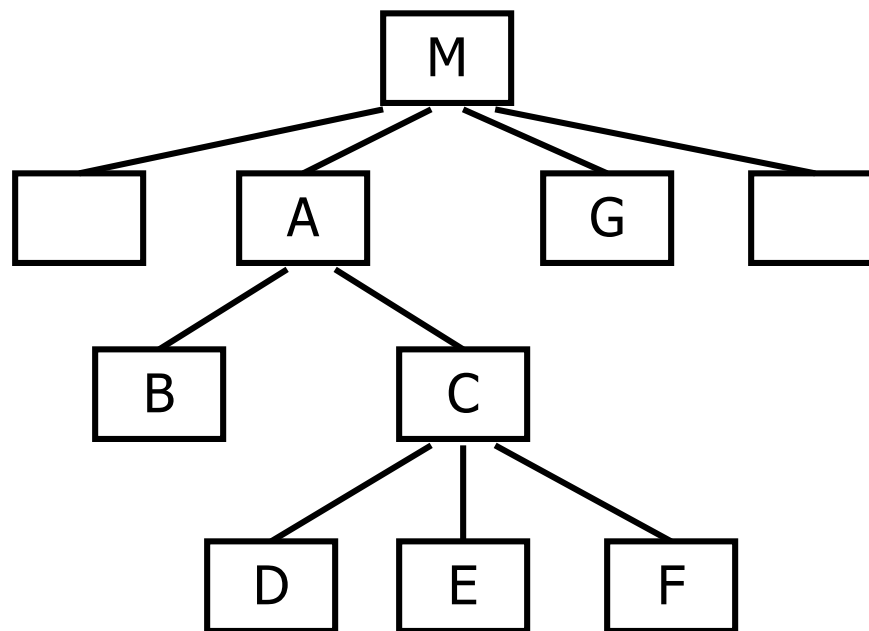


模块设计优化

(5) 模块的作用域应该在控制域之内

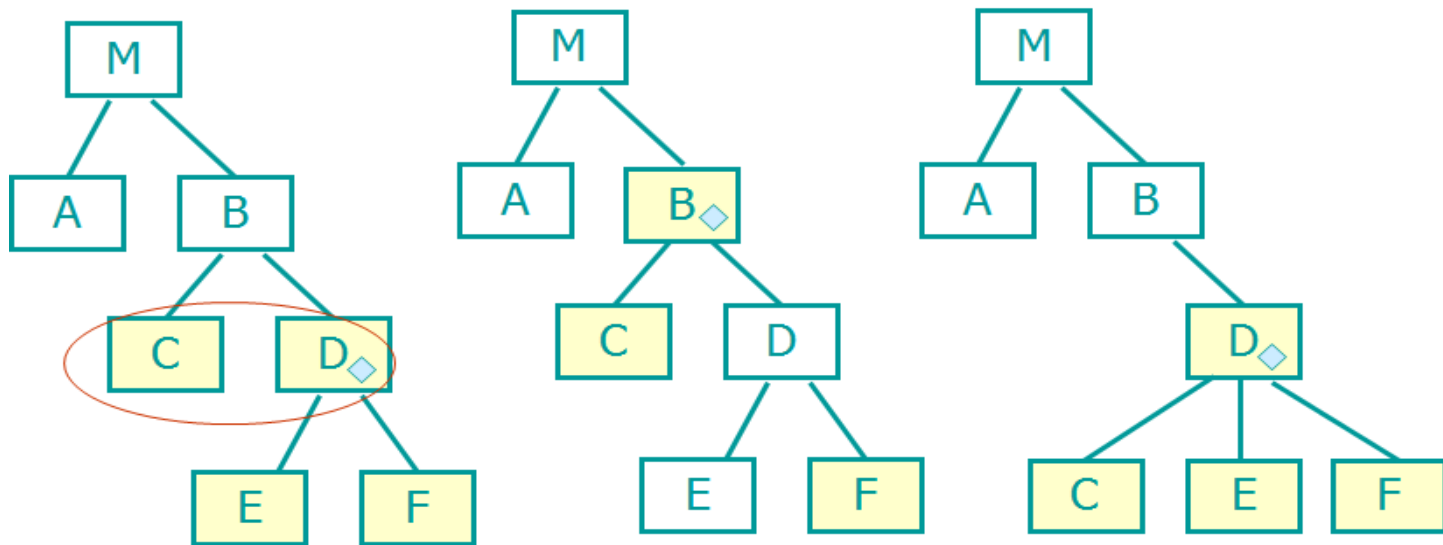
模块的作用域--受该模块内一个判定影响的所有模块的集

模块的控制域--模块本身以及所有直接或间接从属于它的模块的集合。



模块的作用域和控制域

模块设计优化



- 假设C受D中判断的影响
- 在设计过程中，发现模块的作用范围不在其控制范围之内，可以用以下方法改进：上移判定点； 或下移受判定影响的模块。

模块设计优化

- (6) 力争降低模块接口的复杂程度

模块接口复杂是软件发生错误的一个重要原因。

如：求一元二次方程 的根 的模块：

`root (tbl, x)`

说明： 数组tbl是方程的系数， 数组x回送根；

但是，以数组tbl作为参数增加了接口复杂度！

`root (a,, b, c, r1, r2)`

说明： a,, b, c是方程的系数，

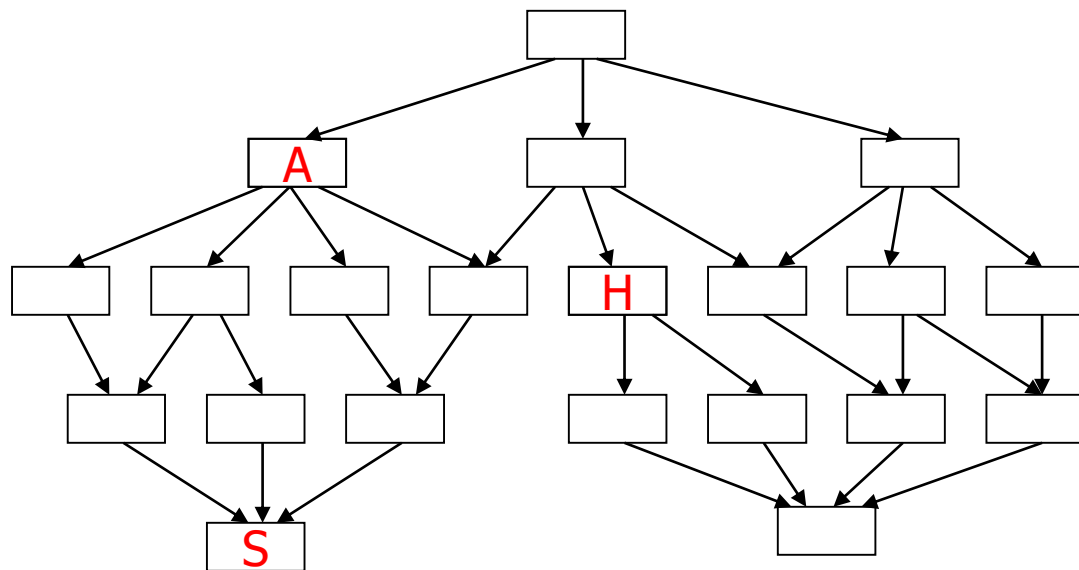
r1, r2是计算的两个根；

模块设计优化

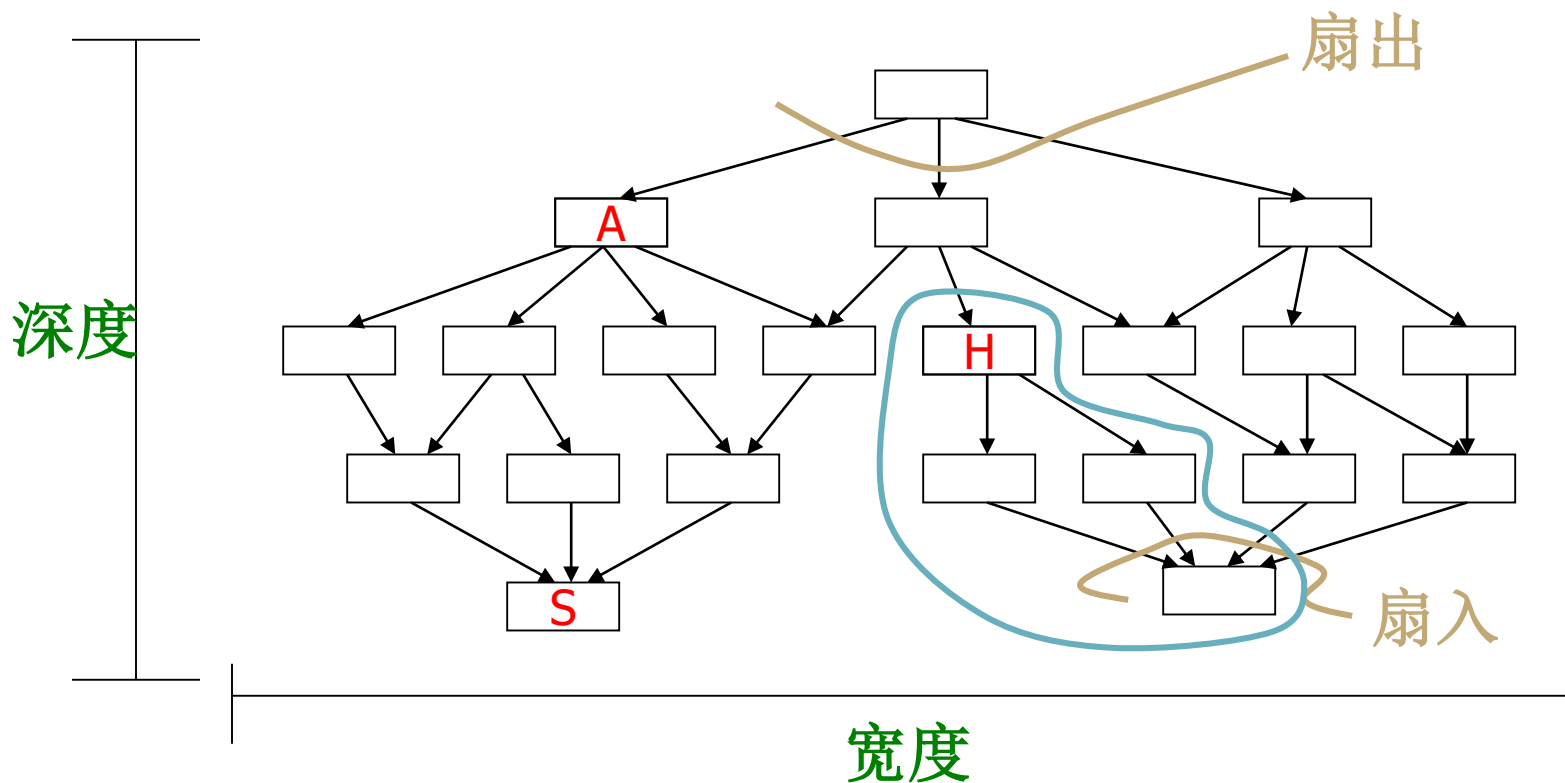
思考题：

在软件结构图中：

1. 软件结构的深度？ 宽度？
2. 模块S的扇入？ 模块A的扇出？
3. 模块H的控制域？



模块设计优化



1. 软件结构的深度为5，宽度为8。
2. 模块S的扇入为3，模块A的扇出为4。
3. 模块H的控制域：H本身及下层3个模块。

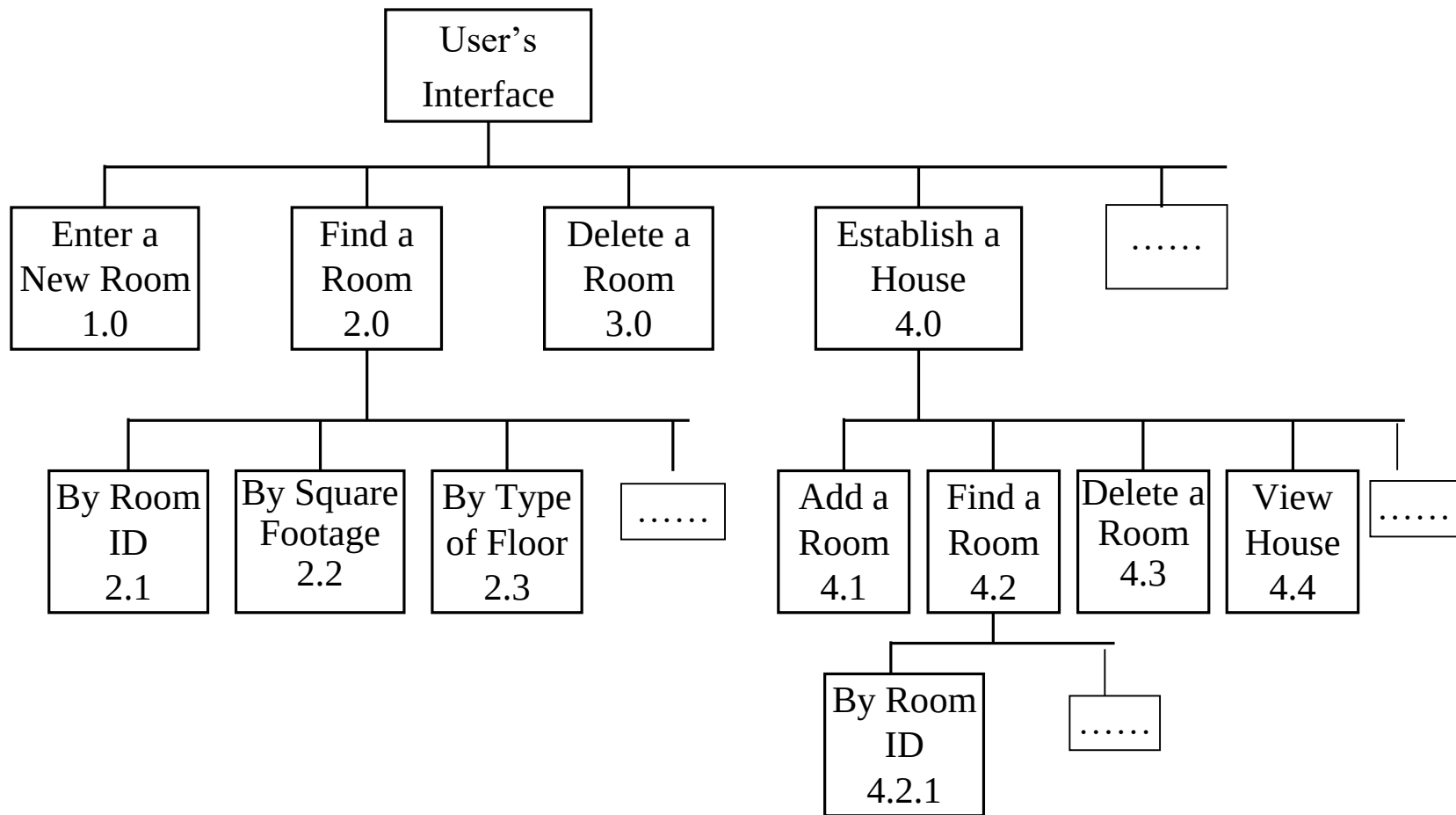
6.4 总体设计工具

6.4.1 HIPO图

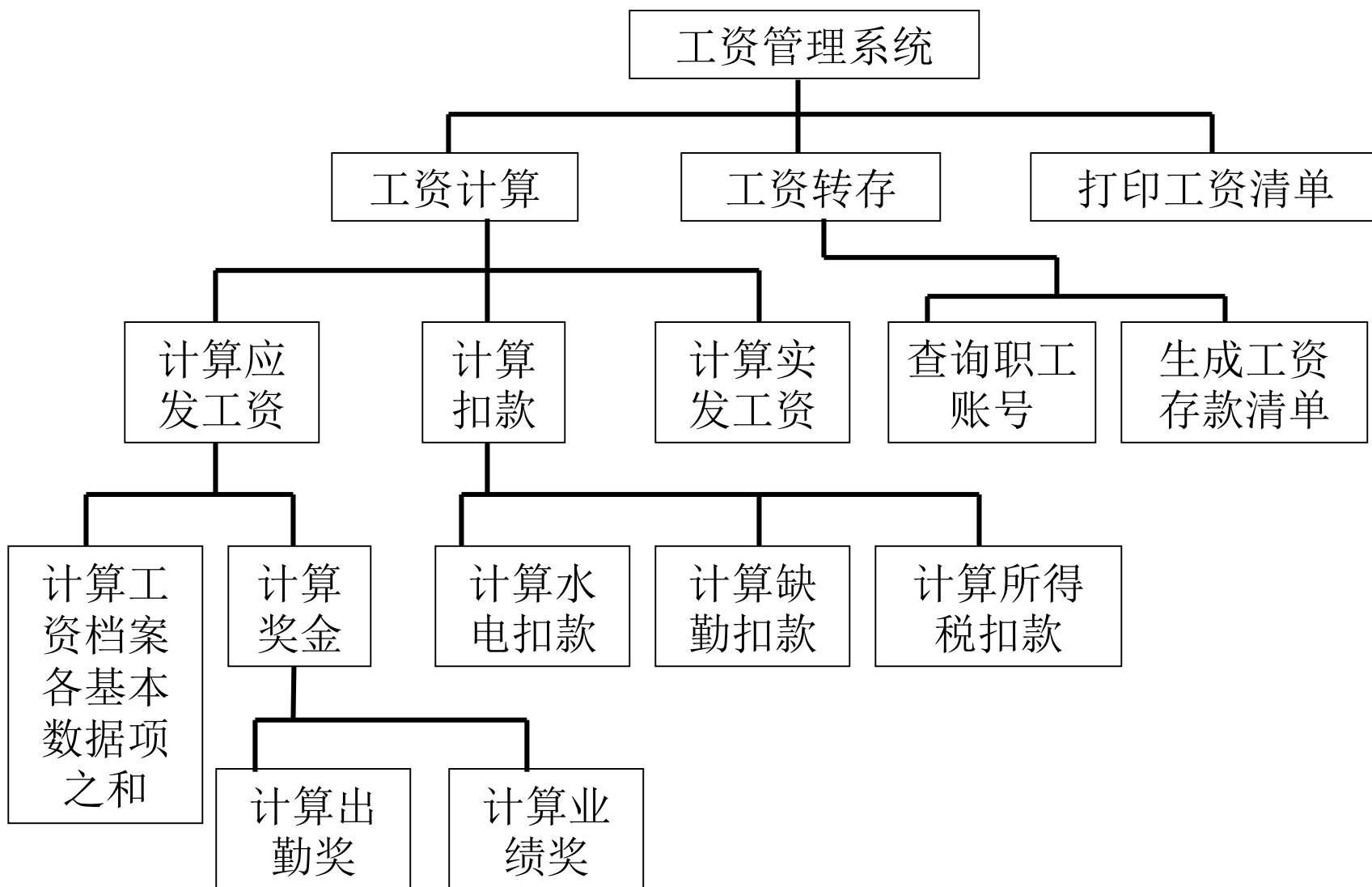
- HIPO图既可以描述软件总的功能（模块）层次结构——H图（层次图），又可以描述每个模块输入/输出数据、处理功能及模块调用的详细情况——IPO图。
- HIPO图以模块分解的层次性以及模块内部输入、处理、输出三大基本部分为基础建立的。矩形代表模块，连线代表调用

1. H图

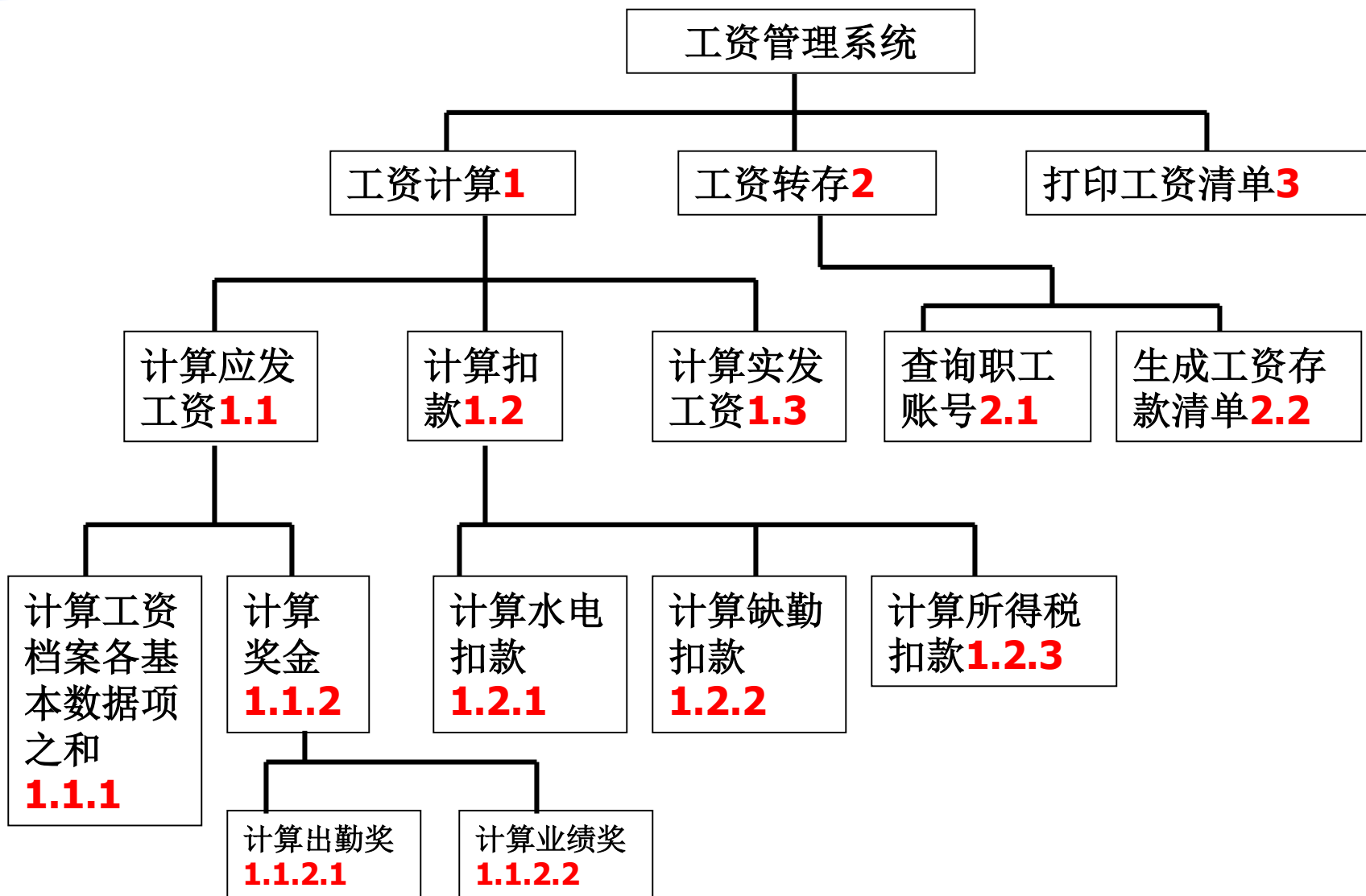
- 层次图可以对除最顶层的方框外的所有方框加编号（编号规则同数据流图）



H图

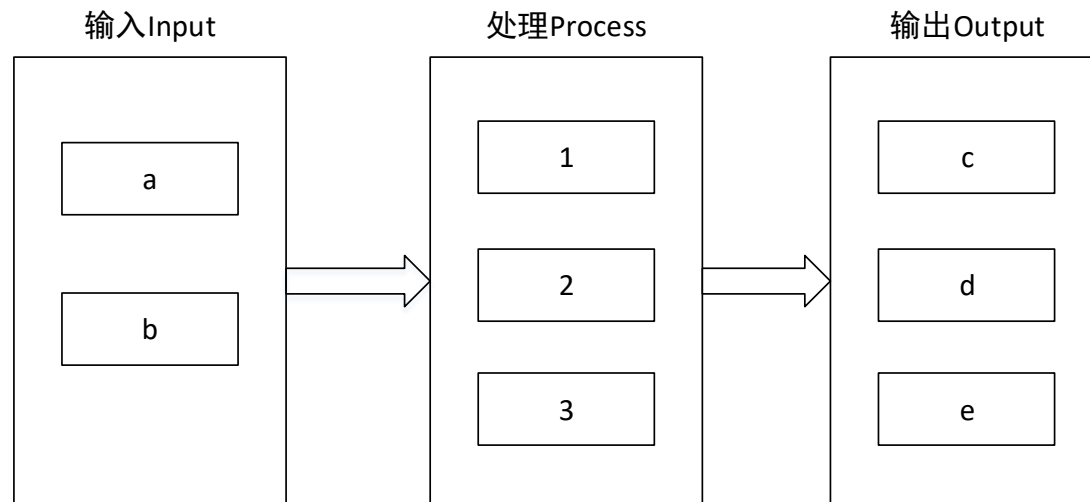


H图



2. IPO图

- H图只说明了软件系统由哪些模块组成及其控制层次结构，并未说明模块间的信息传递及模块内部的处理。因此对一些重要模块还可以绘出IPO (Input / Processing / Output) 图。



IPO表

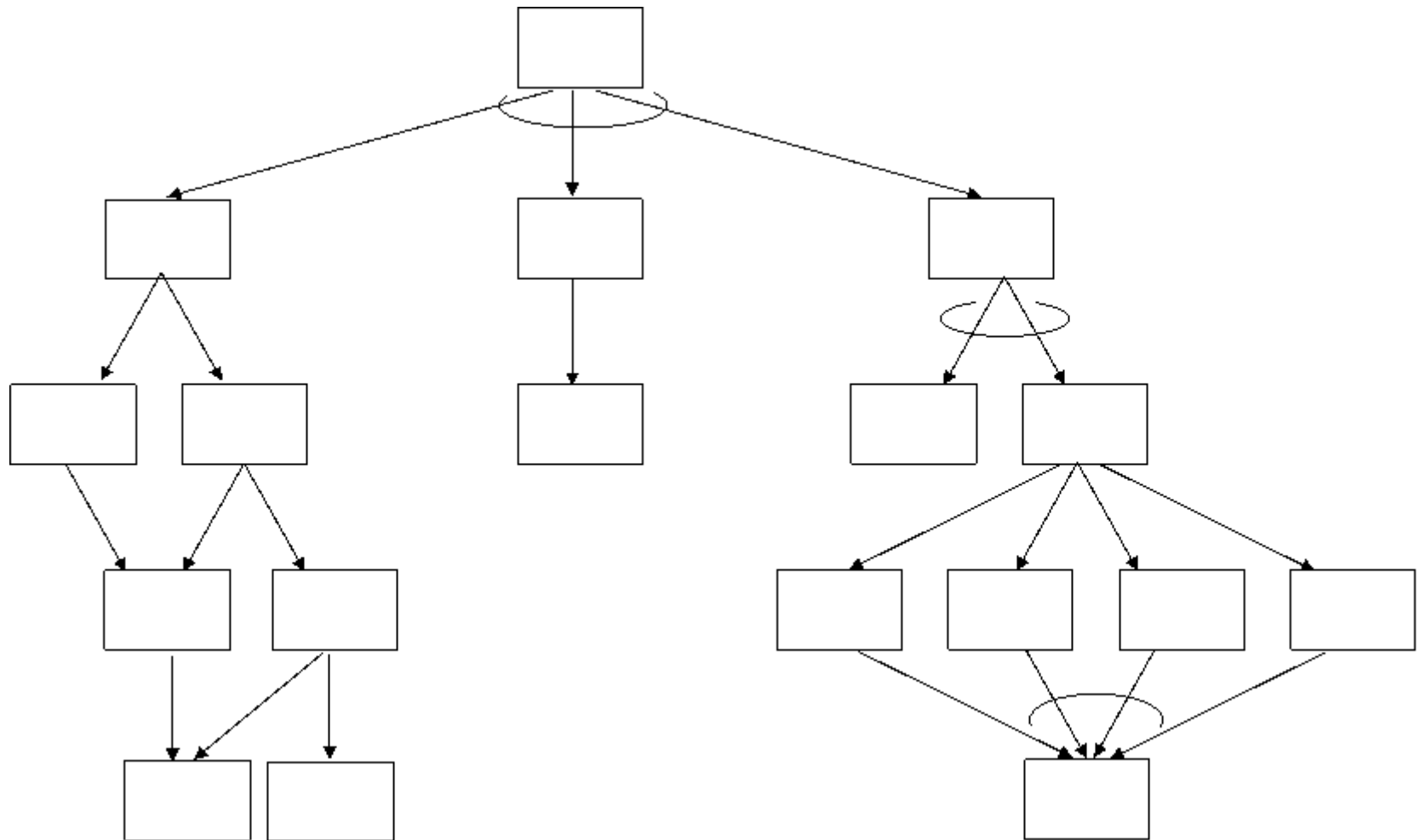
IPO图仅能模块的表示输入和输出，有时一个任务的完成需要调用其他模块，IPO图则不能表示这种关系。为此人们设计了改进的IPO表，

IPO表	
系统：_____	作者：_____
模块：_____	日期：_____ 编号：_____
被调用：	调用：
输入：	输出：
处理：	
局部数据元素：	注释：

6.2.4 系统结构图

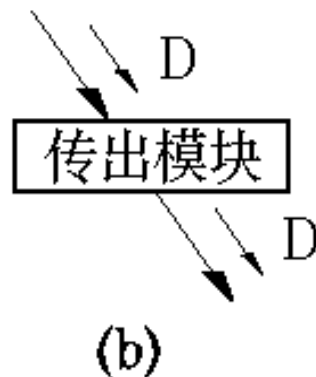
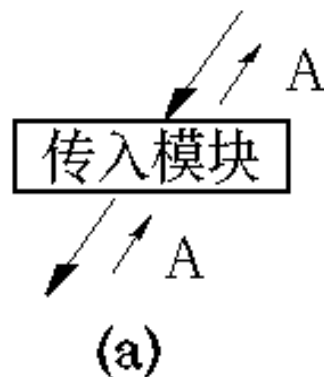
- 系统结构图（软件结构图）表示了一个系统（或功能模块）的层次分解关系，模块之间的调用关系，以及模块之间数据流和控制流信息的传递关系，它是描述系统物理结构的主要图表工具。
- 系统结构图反映的是系统中模块的调用关系和层次关系，谁调用谁，有一个先后次序（时序）关系，所以系统结构图既不同于数据流图，也不同于程序流程图。
- 系统结构图是对软件系统结构的总体设计的图形显示。

系统结构图



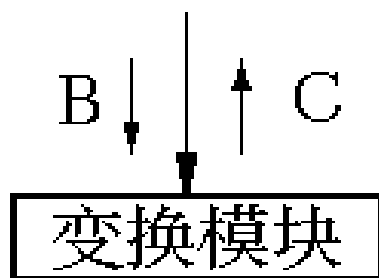
1. 系统结构图中的模块

- **传入模块**--从下属模块取得数据，经过某些处理，再将其传送给上级模块。它传送的数据流叫做逻辑输入数据流。
- **传出模块**--从上级模块获得数据，进行某些处理，再将其传送给下属模块。它传送的数据流叫做逻辑输出数据流。

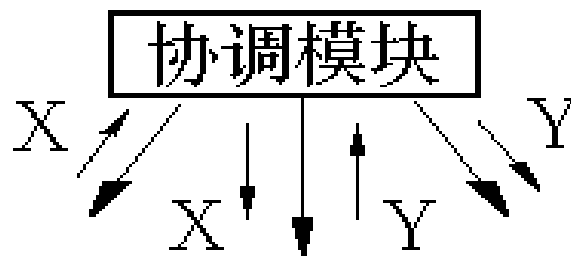


系统结构图中的模块

- **变换模块**--它从上级模块取得数据，进行特定的处理转换成其它形式，再传送回上级模块。它加工的数据流叫做变换数据流。
- **协调模块**--对所有下属模块进行协调和管理的模块。



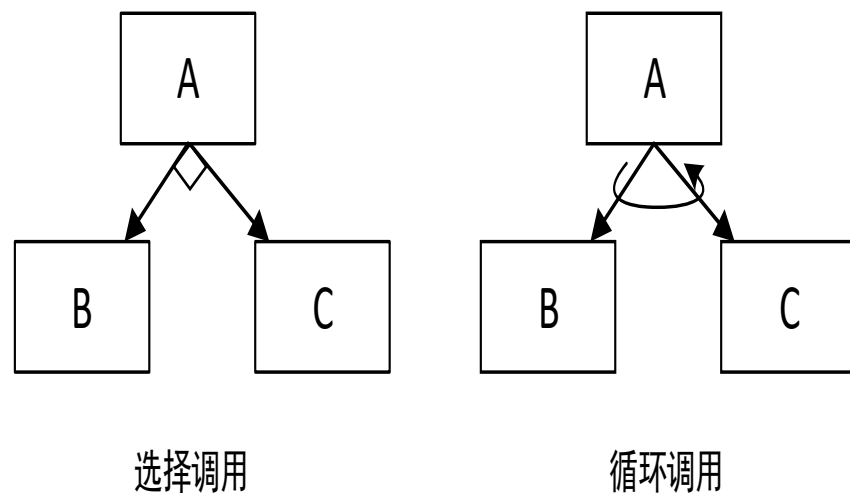
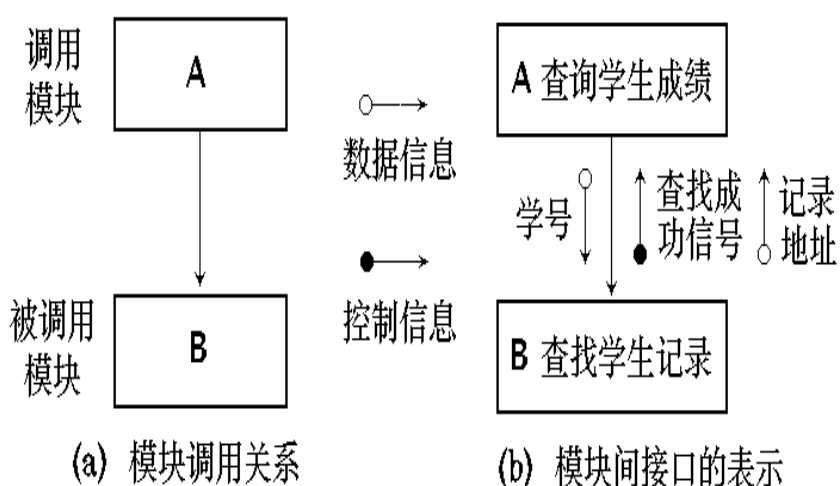
(c)



(d)

系统结构图中的模块调用关系

- 模块间的调用是采用有向线从调用模块指向被调用模块。在模块间调用的有向线上可以携带数据信息和控制信息，称为模块间接口的表示。在一些系统结构图可以省略数据信息和控制信息。



6.5 结构化总体设计

6.5.1 数据流类型

- 结构化设计方法用系统结构图来表达程序模块之间的关系。由于数据流图和系统结构图之间有着一定的联系，因此结构化设计方法便可以和需求分析中采用的结构化分析方法(SA)很好衔接。
- 基于数据流图的结构化设计方法的关键是确定数据流的类型，数据流的类型分变换型数据流和事务型数据流。

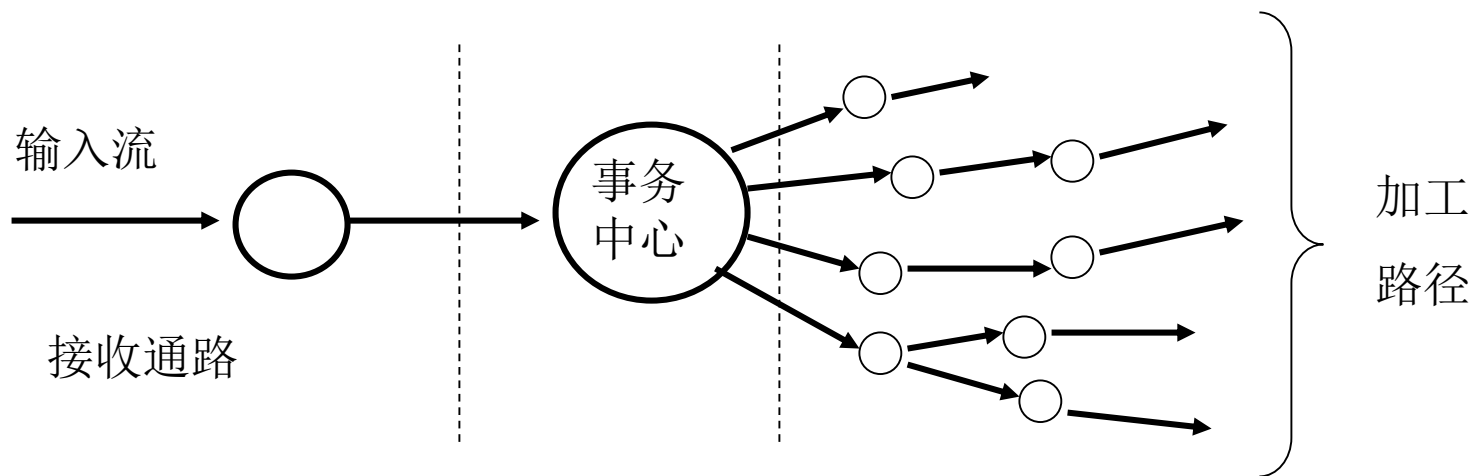
1. 变换型数据流图

- 软件系统的核心是对输入系统的数据进行一系列加工处理，再产生输出数据，这一过程可看做是变换处理。
变换型数据流的工作过程大致分为三步，即取得数据，变换数据和输出数据。



2. 事务型数据流

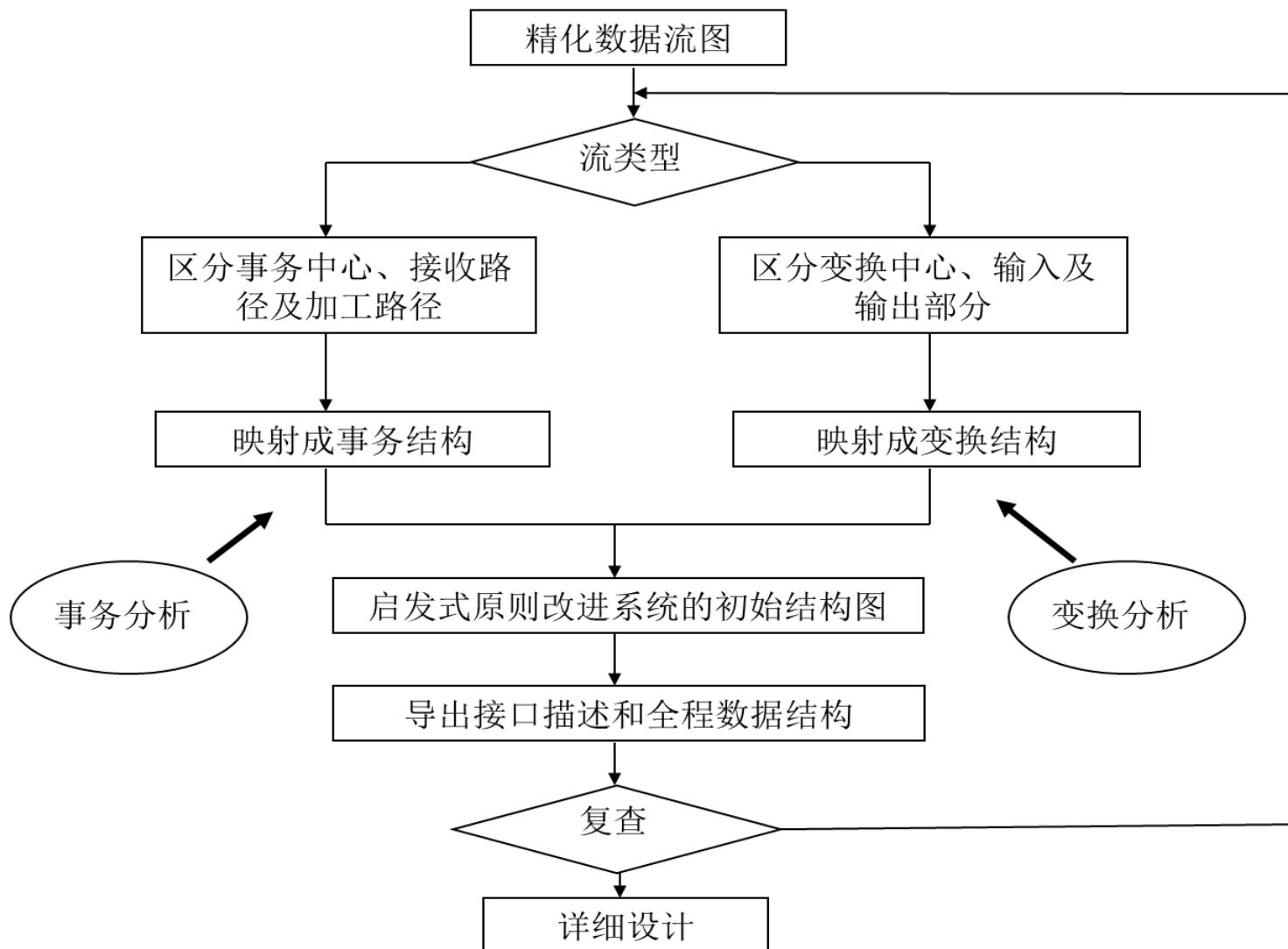
- 事务型数据流的特点是存在一个事务中心，根据事务类型从多个加工路径中选择一个加工路径，然后给出结果。



6.5.2 结构化设计方法

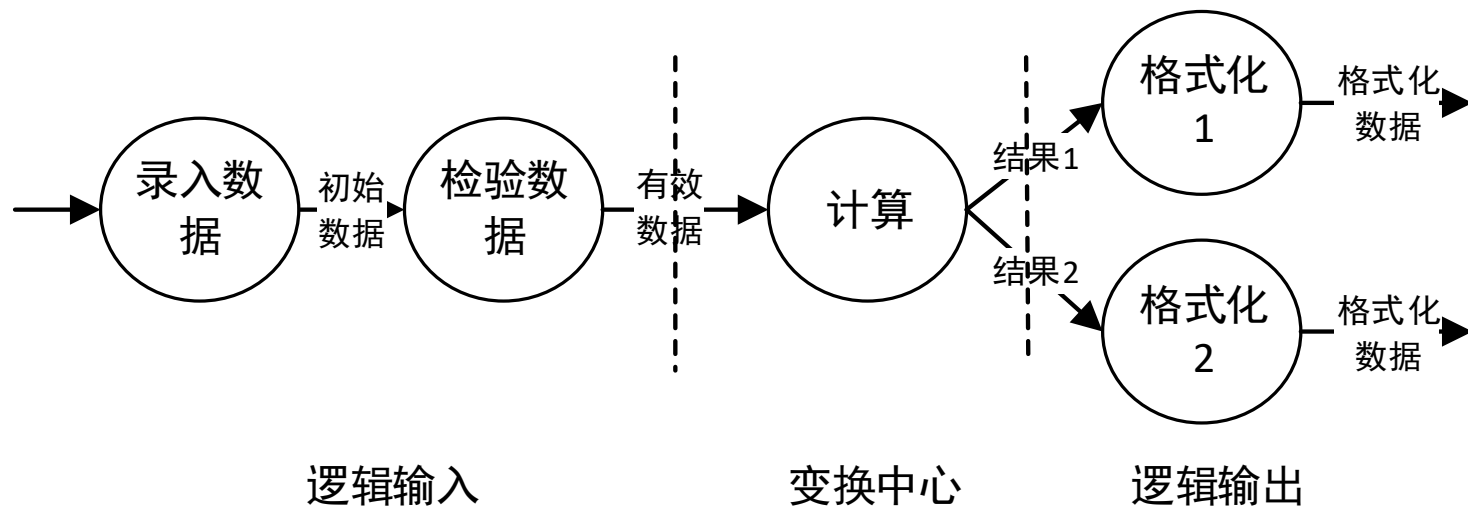
- **首先研究、分析和审查数据流图。**从软件的需求规格说明中弄清数据流加工的过程，检查有无遗漏或不合理之处，对于发现的问题及时解决并修改。
- **根据数据流图决定问题的类型。**数据处理问题典型的类型有两种，分别是变换型和事务型，针对两种不同的类型分别进行分析处理。
- **由数据流图推导出系统的初始结构图。**
- 利用一些启发式原则来**改进系统的初始结构图**，直到得到符合要求的结构图为止。
- 描述模块功能、接口及全局数据结构，修改和补充数据字典。
- 复查，如果出现错误，转入第二步修改完善，否则进入详细设计，同时制定测试计划。

结构化设计过程



6.5.3 变换流映射

- 变换型数据处理问题的过程大致分为三步，即取得数据，变换数据和给出数据。
- 相应于取得数据、变换数据、给出数据，变换型系统结构图的第一层由输入、中心变换和输出等三部分组成。

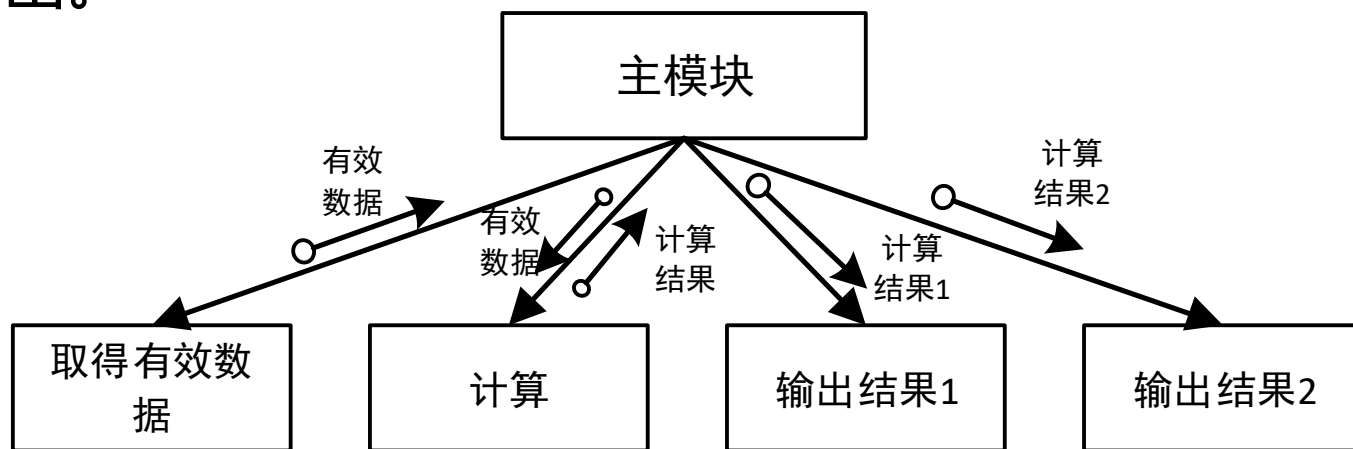


变换流映射

- 一个变换型数据流其变换流设计包括以下步骤：
- 第一步：确定数据流图中的变换中心，逻辑输入、逻辑输出
- 第二步：进行一级分解，设计顶层和第一层模块。

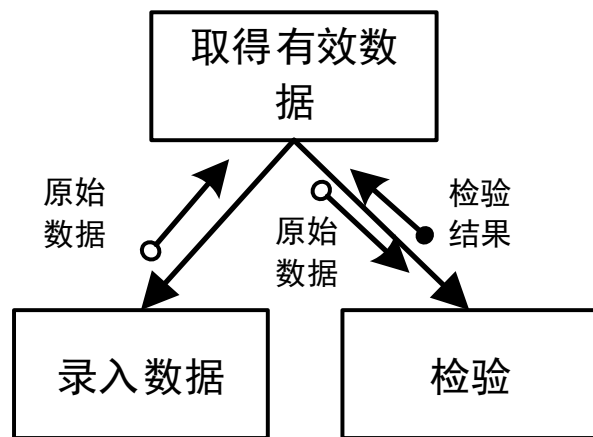
变换流映射

- 系统结构第一层的设计方针：为每一个逻辑输入设计一个输入模块，它为主模块提供数据；为每一个逻辑输出设计一个输出模块，它将主模块提供的数据输出；为中心变换设计一个变换模块，它将逻辑输入转换成逻辑输出。

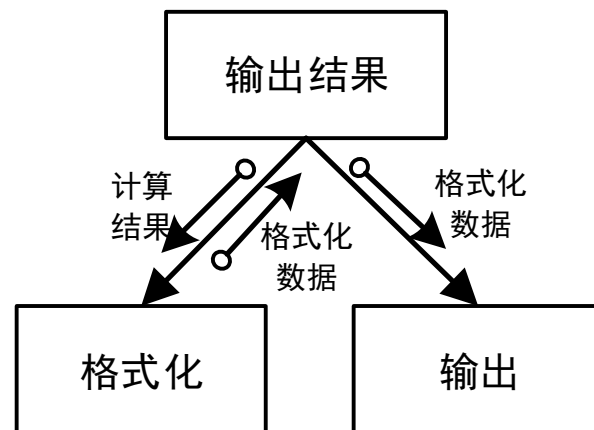


变换流映射

- 第三步：进行二级分解，设计中、下层模块。
- 这一步工作是自顶向下，逐层细化，为每一个输入模块、输出模块、变换模块设计它们的下属模块。
- 输入模块要向调用它的上级模块提供数据，因而它必须有**两个下属模块**：一个是接收数据；另一个是把这些数据变换成它的上级模块所需的数据。



(a) 取得有效数据的下属模块

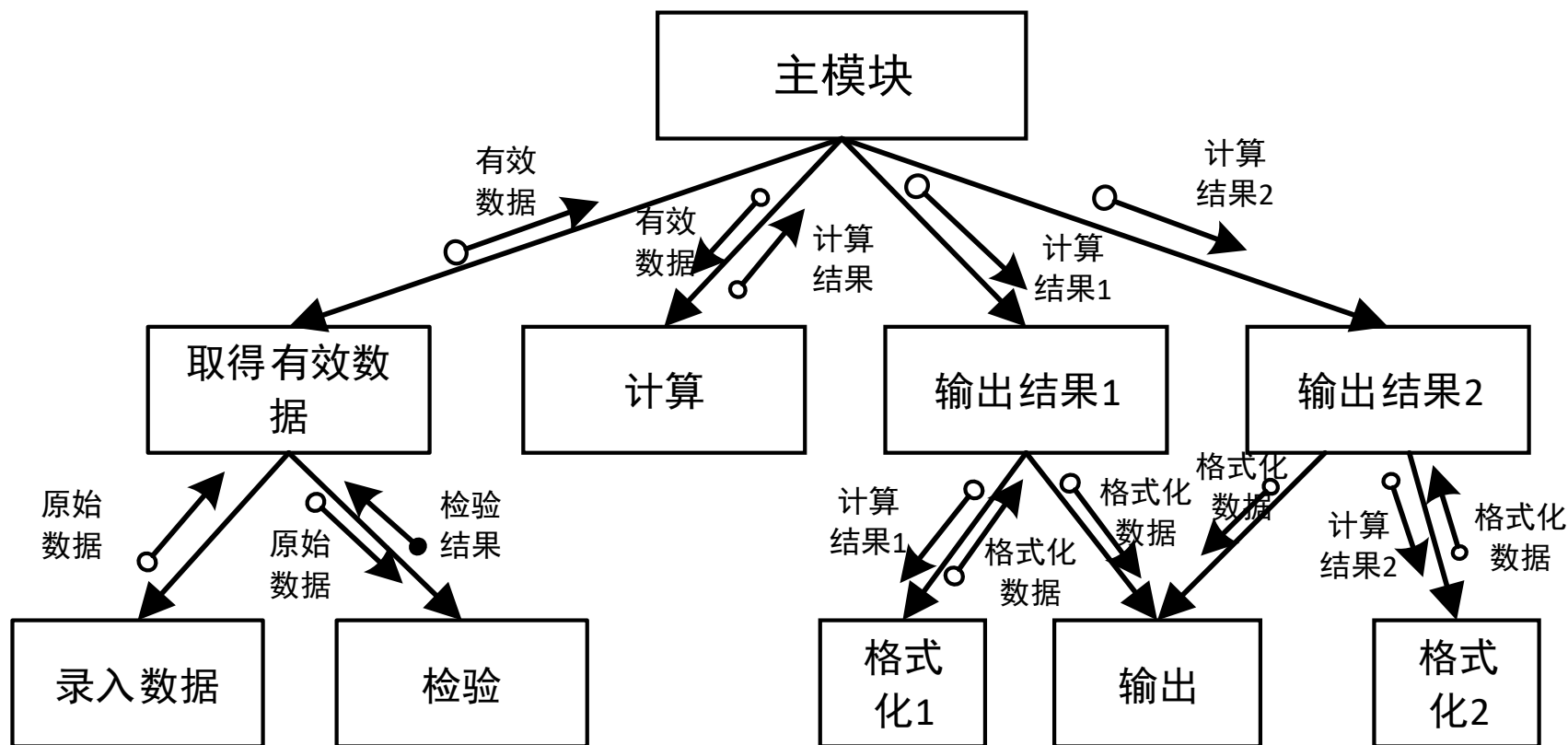


(b) 输出结果的下属模块

变换流映射

- 第四步：设计优化
 - (1) 输入部分优化。对每个物理输入设置专门模块，以体现系统的外部接口，其它输入模块并非真正输入，当它与转换数据的模块都很简单时，可将它们合并成一个模块。
 - (2) 输出部分优化。为每个物理输出设置专门模块，同时注意把相同或类似的物理输出模块合并在一起，以降低耦合度，提高初始结构图的质量。
 - (3) 变换部分优化。根据设计准则，对模块进行合并或调整。

变换流映射



6.5.4 事务流映射

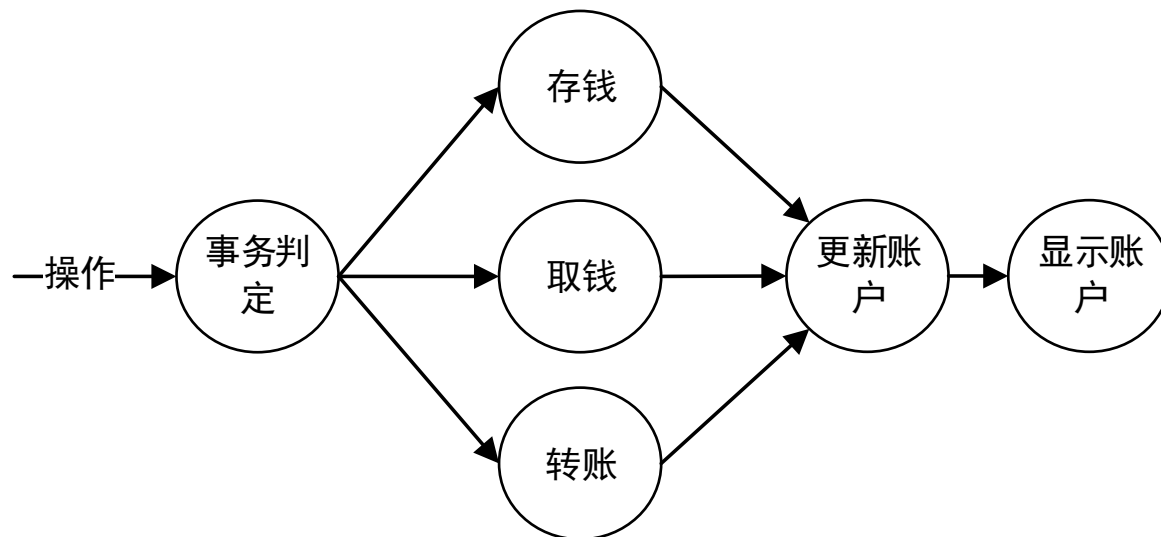
- 事务流设计的要点是把事务流映射成包含一个接收分支和一个事务处理分支的软件结构。
- 接收分支的映射方法和变换流设计映射中输入结构的方法相似，即从事务中心的边界开始，把沿着接收流通路的处理映射成一个模块。事务处理分支结构包含了一个事务中心模块（或调度模块）和它下层的各个动作模块。

事务中心

- 事务中心模块按所接受的事务的类型，选择某一个事务处理模块执行。
- 各个事务处理模块是并列的，依赖于一定的选择条件，分别完成不同的事务处理工作。
- 数据流图的每一个事务流路径应映射成与其自身信息流特征相一致的结构。

事务分析

- ATM机上的软件是一个典型的具有事务流特征的程序。用户在输入银行卡密码后，可以选择下一步的操作，ATM机会根据用户的选择，执行一系列银行业务服务，如存款、取款、转账、查询等。下面设计ATM机软件，其局部数据流图



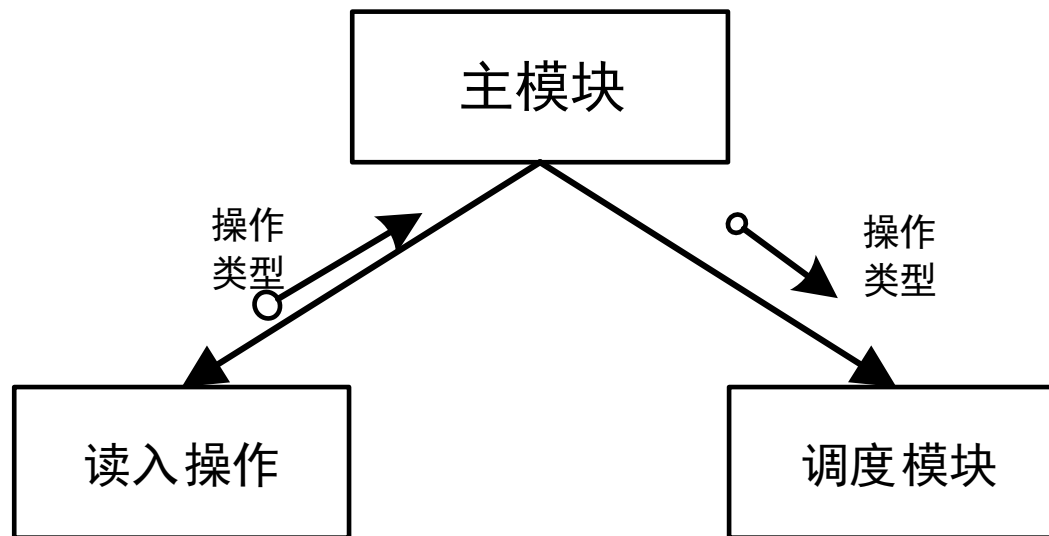
事务流映射

- 第一步：识别事务源和事务中心。
- 利用数据流图和数据词典，从问题定义和需求分析的结果中，找出各种需要处理的事务。通常，事务来自物理输入装置。
- 事务中心通常位于几条操作路径的起始点上，可以从数据流图上直接找出来。输入路径必须与其它所有操作路径区分开来。

事务流映射

- 第二步：将数据流图映射到事务型系统结构图上。

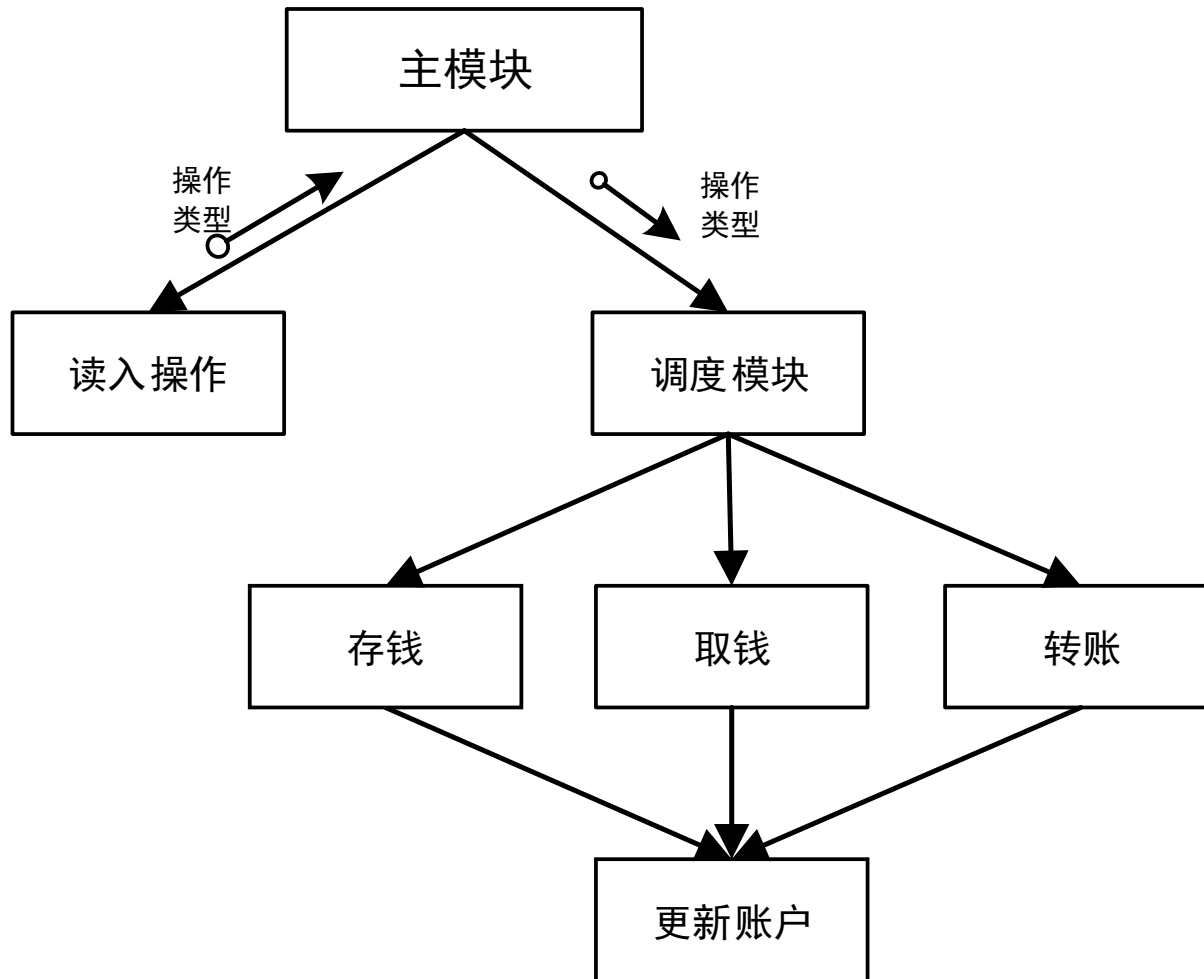
事务流应映射到包含一个输入分支和一个分类处理事务分支的程序结构上。事务处理分支结构包含一个调度模块，它调度和控制下属的操作模块。



事务流映射

- 第三步：分解和细化该事务结构和每一条操作路径的结构。每一条操作路径的数据流图有自己的信息流特征，可以是变换流也可以是事务流。与每一条操作路径相关的子结构可以依照前面介绍的设计步骤进行开发。
- 第四步：利用一些启发式原则来改进系统的初始结构图。

事务分析过程



6.5.5 混合结构分析

- 一般而言，一个大型的软件系统是变换型结构和事务型结构的混合结构。通常利用以变换分析为主，事务分析为辅的方式进行软件结构设计。
- 在系统结构设计时，首先利用变换分析方法把软件系统分为输入、中心变换和输出三个部分，设计上层模块，即主模块和第一层模块，然后，根据数据流图各部分的结构特点，适当地利用变换分析或事务分析，即得到初始系统结构图的一个方案。